EXECUTION-AWARE LIMIT ORDER BOOK FORECASTING UNDER TEMPORAL DISTRIBUTION SHIFT

TECHNICAL REPORT

Alexey Peregudin
School of Electrical and Electronic Engineering
University of Sheffield, United Kingdom
prof.peregudin@gmail.com

June 2026

ABSTRACT

Short-horizon forecasting on limit order book data is methodologically demanding: useful signal is weak, temporal distribution shift is the rule, and even a genuine statistical edge can fail to survive execution costs. This report studies those challenges on a deliberately small scale, using single-symbol BTCUSDT futures on twelve first-of-month snapshot days run on home-compute resources, with the aim of demonstrating research discipline rather than impressing by scale.

The evaluation covers short-horizon mid-price direction, return, and execution-relevant quantities using an expanding monthly walk-forward ($\sim$1.6M event rows, three held-out months). Strong baselines (an imbalance rule, linear models, and gradient-boosted trees) are compared against a compact causal temporal convolutional network and a multi-task variant with calibrated return quantiles and adverse-selection heads, optionally conditioned on a latent state-space context. Forecasts are assessed both statistically and through a taker-cost backtest and a passive market-making simulator.

The predictive findings are modest: short-horizon directional signal exists and holds across the held-out months; a compact TCN leads at the shortest horizon; a latent state-space context, rather than added capacity, is the most effective architectural change ($+9.7$ points at $h{=}10$). None of it survives execution. No model clears a 5 bps taker fee, and no quoting policy robustly beats not quoting, because adverse selection dominates spread capture at these horizons and costs. We report this negative execution result as the main finding.

Keywords Limit order book · Market microstructure · Deep learning · Temporal convolutional network · Temporal distribution shift · Walk-forward validation · Quantile calibration · Passive market making · Adverse selection · Execution-aware forecasting

Contents

1 Introduction **4**
1.1 Motivation: predictive skill is not decision value . . . 4
1.2 Research question and scope . . . 4
1.3 Contributions . . . 5
1.4 Report roadmap . . . 6

2 Background and Related Work **6**

3 Problem Formulation **6**
3.1 The order-book process and event time . . . 7
3.2 Forecasting targets . . . 7
3.3 Temporal distribution shift . . . 7
3.4 Decision value . . . 8

4 Data, Microstructure, Features and Labels **8**
4.1 The raw data . . . 8
4.2 Order-book reconstruction and event-time sampling . . . 9
4.3 Microstructure features . . . 9
4.4 Causal, per-day feature computation . . . 11
4.5 Regime descriptors and causal bucketing . . . 11
4.6 Labels and their class structure . . . 11
4.7 Markout and adverse-selection targets . . . 12
4.8 Observability constraints . . . 12

5 System Architecture and Leakage Discipline **13**
5.1 End-to-end dataflow . . . 13
5.2 Repository and module organisation . . . 13
5.3 Data contracts . . . 14
5.4 Leakage discipline: the invariants . . . 14

6 Models **15**
6.1 Model interface and registry . . . 15
6.2 Baselines . . . 16
6.3 The direction-only TCN . . . 16
6.4 The execution-aware multi-task TCN . . . 16
6.5 Return-head ablation variants . . . 17
6.6 Latent state-space context . . . 18

7 Experimental Protocol and Evaluation **18**
7.1 Two evaluation campaigns . . . 18
7.2 Expanding walk-forward protocol . . . 18
7.3 Datasets and split sizes . . . 18
7.4 Evaluation metrics . . . 19
7.5 Model selection and two-phase training . . . 19
7.6 Robustness reporting . . . 20

8 Predictive Results **20**
8.1 Direction accuracy across held-out months . . . 20
8.2 Return signal . . . 21
8.3 Distributional calibration . . . 22
8.4 Execution-head accuracy and summary . . . 22

9 From Prediction to Execution **23**
9.1 Taker sanity backtest . . . 23
9.2 The passive market-making simulator . . . 24
9.3 Policies . . . 25
9.4 Market-making results . . . 25
9.5 The control optimiser and the anti-triviality check . . . 25

10 Ablations and Development **26**
10.1 Return-head ablation . . . 27
10.2 Latent state-space context . . . 27
10.3 Development: regularisation, early stopping, and the calibration arc . . . 27
10.4 Queue-aware partial-fill model . . . 27

11 Engineering, Reproducibility, and Testing **28**
11.1 Performance and scale . . . 28
11.2 Reproducibility and run artefacts . . . 28
11.3 Crash-resumability . . . 29
11.4 Testing discipline . . . 29

12 Discussion, Limitations and Conclusion **29**
12.1 Predictive skill exists, narrowly . . . 30
12.2 The return head was the right thing to remove . . . 30
12.3 Calibration is trainable, but was fragile . . . 30
12.4 Where the signal fails in execution . . . 30
12.5 Methodological takeaways . . . 30
12.6 Limitations . . . 31
12.7 Future work . . . 31
12.8 Conclusion . . . 31

A Feature and Label Details **32**

B Module Architecture and Data Contracts **32**

C Full Results **32**

D Hyperparameters and the Development Campaign **32**

Notation

Symbol	Meaning
Indices and dimensions	
t	event-time index within a day (after striding)
d	trading day (a first-of-month snapshot)
$k,\ K$	book level and number of levels ($K = 5$)
$h,\ \mathcal{H}$	forecast horizon in events; $\mathcal{H} = \{10, 50, 200\}$
L	input sequence length ($L = 100$)
Book and microstructure (Section 4)	
$p_t^{b,k}$, $q_t^{b,k}$	bid price and size at level k
$p_t^{a,k}$, $q_t^{a,k}$	ask price and size at level k
m_t, s_t, s_t^{rel}	mid-price, spread, relative spread
μ_t	microprice (size-weighted mid)
$I_t^{(1)}$, $I_t^{(K)}$	level-1 and depth-weighted imbalance
$\mathrm{OFI}_{t,L}$	trailing order-flow imbalance
$\sigma_{t,W}$	realised volatility over window W
Forecasting targets (Sections 3.2, 4)	
$r_{t,h}$	forward mid-price return
$y_{t,h}^{\mathrm{dir}}$	direction class in $\{-1, 0, +1\}$
$\varepsilon_h,\ \alpha$	direction threshold and its scale ($\alpha = 0.5$)
$\hat{Q}_\tau$	predicted return quantile, $\tau \in \{0.05, 0.5, 0.95\}$
$M_{t,h}^{\mathrm{side}}$, $A_{t,h}^{\mathrm{side}}$	markout and adverse-selection targets
Models (Section 6)	
$\mathbf{X}_{t-L+1:t}$	causal input sequence window
$\mathbf{c}_t$	regime / context vector
$\mathbf{z}_t$	pooled learned representation
$\boldsymbol{\xi}_t$	latent state-space (Kalman) state
$\theta,\ \phi$	encoder and head parameters
$\omega_\bullet$	multi-task loss weights
Execution (Section 9)	
x_t, C_t, W_t	inventory, cash, marked wealth
R_t	per-decision market-making reward
$\lambda_\bullet$	reward penalty coefficients
$\hat{J}_t(a)$	control-optimiser objective for action a

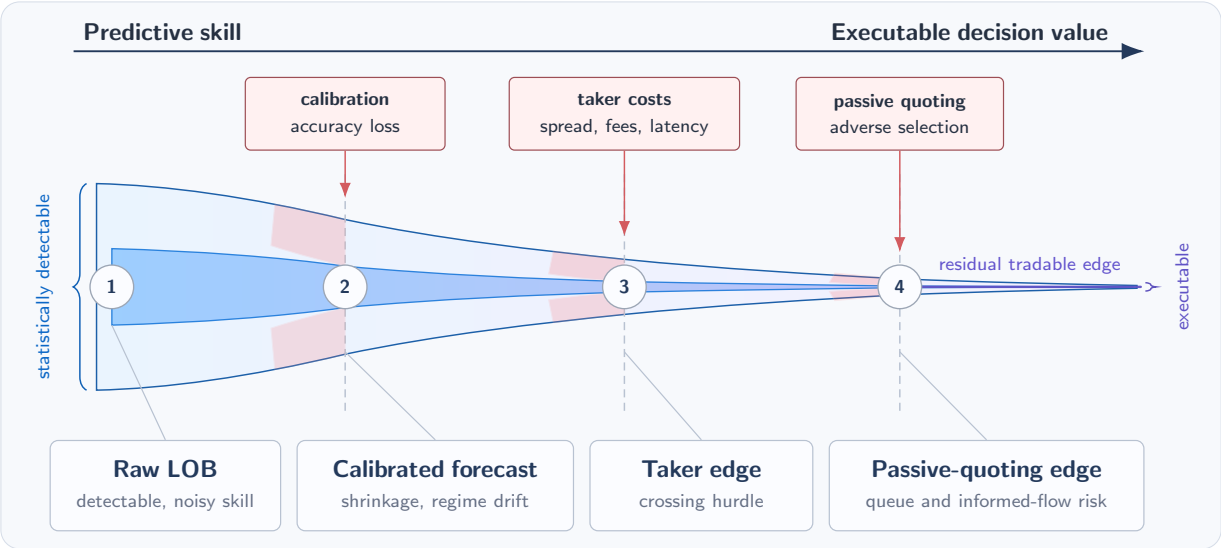


Figure 1: From predictive signal to decision value: a forecast through calibration, execution fees, and adverse selection.

1 Introduction

1.1 Motivation: predictive skill is not decision value

Short-horizon prediction on the limit order book (LOB) is a canonical machine-learning problem adjacent to high-frequency trading, and an awkward one. The raw data is enormous and highly redundant: a top-of-book feed emits a fresh snapshot on every visible change, at sub-second cadence. The quantity one actually wants to predict, the next move of the mid-price, is faint, noisy, and non-stationary. Prices are produced by strategic, adaptive agents rather than by a fixed stochastic process, so a relationship that holds this month may weaken or invert the next. And whatever signal does exist lives at horizons of seconds, where it is exquisitely sensitive to latency and to the cost of crossing the spread.

A large and active literature applies deep sequence models to this problem and reports encouraging numbers: directional accuracy or F_1 on mid-price movement that clears simple baselines. What this literature much more rarely does is ask the question a trading desk actually cares about: *does that statistical edge survive contact with the economics of execution?* A forecast must be turned into an order, and an order pays fees, crosses a spread or waits in a queue, fills only sometimes, accrues inventory risk, and, most corrosively for a liquidity provider, is *adversely selected*: the counterparties who take a passive quote are precisely those who know something the quoter does not.

This tension — between *predictive skill* and *decision value* — is the axis around which the entire report is organised. The two are routinely conflated, and they should not be. A model can be measurably better than chance at calling the next tick and still be worthless once executed, or even worse than standing aside. Figure 1 previews the thesis. As a raw forecast is pushed through calibration, taker costs, and the adverse-selection penalty of passive quoting, the apparent edge narrows at every stage; by the time it reaches a decision, little or nothing remains. The purpose of this work is not to deny that signal exists, but to locate and measure exactly where, and by how much, it is consumed.

1.2 Research question and scope

I restrict the study to one instrument, one venue, and one book depth, observed on a calendar of monthly regime snapshots. The narrow scope keeps the monthly comparisons interpretable and lets the reconstruction, feature, and evaluation steps be reproduced on one machine:

```
symbol:     BTCUSDT only
venue:      Binance-futures (Tardis book_snapshot_5 archives)
LOB depth:  top-5 levels of each side only
calendar:   the first calendar day of each month (regime snapshots)
sampling:   event-time, every 10th book update (event_stride = 10)
horizons:   10, 50, 200 book events  (h50 ~ 500 raw updates ~ tens of seconds)
hardware:   a single workstation (CPU or one consumer GPU)
```

Concretely, the data is Binance-futures BTCUSDT, reconstructed from Tardis `book_snapshot_5` archives to the top five levels of each side and sampled in event time. Horizons are measured in book events, $h \in \{10, 50, 200\}$; after striding, a horizon of 50 events corresponds to roughly 500 raw updates, on the order of tens of seconds. The scope is narrow on purpose. A single liquid symbol observed across *separated* monthly snapshots makes temporal distribution shift explicit and testable, and keeps every fitted transform causal and the pipeline runnable on one workstation. Adding symbols, venues, or continuous history would answer a different question and would make the execution analysis harder to follow, so breadth is left to future work.

The guiding research question is the following.

> Can compact sequence models trained on top-5 BTCUSDT limit order book snapshots produce calibrated, execution-relevant forecasts that remain useful across held-out first-of-month market snapshots, and can those forecasts improve passive market-making decisions under inventory, latency, fill uncertainty, and adverse-selection costs?

This breaks into six concrete sub-questions:

- **Q1** Is there stable short-horizon directional signal beyond simple baselines?
- **Q2** Do deep sequence models outperform linear and gradient-boosted baselines under monthly distribution shift?
- **Q3** Does multi-task learning improve the execution-relevant heads (predictive quantiles, markout, and adverse selection)?
- **Q4** Does the point-return head help or harm the shared representation?
- **Q5** Does a latent state-space context improve robustness?
- **Q6** Does any predictive signal survive taker fees or the economics of passive market making?

These sub-questions are revisited and closed, one by one, in the hypothesis-test summary of the Discussion (Section 12).

1.3 Contributions

This report makes the following contributions.

1. **An end-to-end LOB research platform** that converts raw exchange snapshots into causal features, labels, walk-forward datasets, calibrated forecasts, and execution-aware decision tests, with each stage testable on its own and the causal invariants checked at the stage boundaries.
2. **A compact execution-aware multi-task temporal convolutional network (TCN)** that emits, per horizon, a direction class, a point return, monotone predictive quantiles, and markout and adverse-selection estimates: exactly the quantities a passive-quoting policy consumes.
3. **A twelve-month expanding walk-forward protocol** that tests whether a predictive edge is robust across multiple held-out monthly regimes rather than an artefact of a single test month.
4. **A latent state-space context** (a small linear-Gaussian Kalman filter) that supplies a smoother causal hidden-state estimate and materially improves short-horizon directional accuracy.
5. **An execution stack** comprising a taker-cost sanity backtest, a queue-aware partial-fill market-making simulator, and a transparent control-style quote optimiser, which converts forecasts into decisions and measures their economic value against an explicit do-nothing baseline.
6. **A reproducible, crash-resumable implementation** backed by 267 automated tests and fully self-contained per-run artefacts, engineered to run on a single machine.

7. **A separation of two results.** Short-horizon predictive structure is real and holds across the held-out months, but it does not survive 5 bps taker fees or the adverse selection of passive market making under the simulator used here.

Together these elements form a small analogue of an institutional quantitative-research loop: state a modelling hypothesis, build leak-free data contracts, train under temporal distribution shift, measure predictive accuracy and decision value as *separate* questions, and keep every artefact for re-analysis. The headline outcome is as much about that separation as about any single number:

Headline result. Short-horizon predictive signal exists — a compact TCN slightly but robustly beats the linear baselines on mid-price direction, and the multi-task model produces well-calibrated quantile intervals — but it carries no robust decision value under the execution model: it does not survive 5 bps taker fees, and no forecast-driven passive-quoting policy reliably beats not quoting at all.

1.4 Report roadmap

The remainder of the report follows the path of the data. Section 2 places the work in the literature; Section 3 formalises the order-book process, the forecasting targets, and the two economic objectives. Section 4 describes the data, the microstructure mathematics, and the causal construction of features, regimes, and labels; Section 5 details the system architecture and the leakage invariants the pipeline enforces. Section 6 specifies the models, from baselines to the multi-task TCN and its latent-state context, and Section 7 sets out the walk-forward protocol, the metrics, and the validation-only selection discipline. Sections 8 and 9 report the predictive and the execution results, the report's two central questions, and Section 10 isolates which design choices actually mattered. Section 11 documents the engineering and reproducibility, and Section 12 synthesises the findings against the sub-questions above, states the limitations, and concludes.

2 Background and Related Work

The mechanics and statistical regularities of limit order books are well documented [Gould et al., 2013, Bouchaud et al., 2018], as is the friction this report finds decisive: the classical analysis of Glosten and Milgrom [1985] shows that liquidity providers are systematically adversely selected. Two hand-built signals recur because they work, and we use both: order-flow imbalance, whose near-linear link to short-horizon price moves was established by Cont et al. [2014], and the micro-price, a size-weighted mid that Stoikov [2018] shows is a strong predictor of the future mid. Deep models are now the dominant approach to short-horizon prediction on the book, the convolutional-recurrent DEEPLOB of Zhang et al. [2019] being the standard benchmark; the sobering and repeated finding is that reported predictability is small and fragile once evaluation is made rigorous [Lucchese et al., 2024], which our results echo. For the sequence encoder we adopt a temporal convolutional network [Bai et al., 2018] with dilated causal convolutions [van den Oord et al., 2016]. We keep it small and regularised: on a weakly predictable, non-stationary target a high-capacity model mostly overfits, and the development grid in Section 10 bears this out. The multi-task heads follow Caruana [1997].

The evaluation protocol matters as much as the model, because small amounts of leakage can change the apparent edge. Financial series are non-stationary and naive cross-validation leaks across time, so we follow the walk-forward, leakage-aware procedures catalogued by López de Prado [2018] and quantify uncertainty with a moving-block bootstrap [Politis and Romano, 1994] that preserves short-range autocorrelation. Because an execution policy needs a calibrated distribution rather than a sharper point estimate, the quantile heads are trained with the pinball loss [Koenker and Bassett, 1978] and assessed by interval coverage, a proper-scoring-rule view [Gneiting and Raftery, 2007]; calibration is not automatic in deep networks [Guo et al., 2017], which motivates the dedicated selection score of Section 7. The latent context is a small linear-Gaussian state-space model [Kalman, 1960] fitted by EM [Ghahramani and Hinton, 1996]. Finally, the gap between a good forecast and a profitable quote is the subject of the inventory-control literature [Ho and Stoll, 1981, Avellaneda and Stoikov, 2008]; our market-making test is empirical, not a closed-form control solution: a historical-replay simulator that scores forecast-driven policies against forecast-free baselines and an explicit do-nothing reference, with gradient-boosted trees [Ke et al., 2017] as a non-linear tabular baseline. I combine these ingredients in one pipeline — LOB features, a sequence model, walk-forward evaluation, and a replay-based execution test — so that forecasting and execution can be measured separately. At these horizons and costs, the first does not imply the second.

3 Problem Formulation

I separate two objects throughout the report. The first is the forecast target, measured by classification and calibration metrics; the order-book process and these targets are defined below. The second is the economic outcome after those forecasts are turned into orders, formalised here as two functionals and measured in Section 9. A forecast may score well on the first and still fail the second, which is why they are kept formally distinct. The full notation index appears on the dedicated page immediately after the table of contents.

3.1 The order-book process and event time

At each retained observation the venue exposes the top $K = 5$ levels of each side of the book. We collect them, best-first on each side, into the *book state*

$$\mathbf{b}_t = \big(p_t^{b,k},\, q_t^{b,k},\, p_t^{a,k},\, q_t^{a,k}\big)_{k=1}^{K}, \qquad K = 5. \tag{1}$$

The instantaneous reference price is the mid-price

$$m_t = \tfrac{1}{2}\big(p_t^{a,1} + p_t^{b,1}\big), \tag{2}$$

from which every forecasting target is derived. The remaining microstructure quantities (spread s_t, relative spread s_t^{rel}, microprice μ_t, imbalance, order-flow imbalance, and realised volatility) are defined where they are first used, in Section 4.

The raw feed records a row on *every* change to the top of the book, at sub-second cadence and with heavy redundancy. We therefore sample in *event time*, retaining every `event_stride`-th update (here every tenth). This yields a discrete index $t = 1, 2, \ldots, T_d$ within each day d and turns the horizon into a meaningful forecasting granularity: a horizon of $h = 50$ events corresponds to roughly 500 raw updates, on the order of tens of seconds. Throughout, horizons are $\mathcal{H} = \{10, 50, 200\}$ events.

Each first-of-month day is treated as an *independent episode*. No feature window, label, or holding period may cross a day boundary; this calendar-discontinuity invariant is enforced pipeline-wide (Sections 4 and 5). It is what lets each monthly snapshot be treated as a separable regime instead of one contiguous series.

3.2 Forecasting targets

Given the history up to event t, the model predicts, for each horizon $h \in \mathcal{H}$, three coupled quantities derived from the future mid-price.

Return. The forward mid-price return is

$$r_{t,h} = \frac{m_{t+h} - m_t}{m_t}. \tag{3}$$

Direction. The signed move, with a neutral band, is

$$y_{t,h}^{\mathrm{dir}} = \begin{cases} +1, & r_{t,h} > \varepsilon_h, \\ 0, & |r_{t,h}| \le \varepsilon_h, \\ -1, & r_{t,h} < -\varepsilon_h, \end{cases} \qquad \varepsilon_h = \alpha \cdot \mathrm{median}_{\mathrm{train}}\big(s_t^{\mathrm{rel}}\big), \quad \alpha = 0.5. \tag{4}$$

The neutral band $|r_{t,h}| \le \varepsilon_h$ is scaled to the typical relative spread, so a move counts as directional only once it clears the cost of crossing the book. The threshold ε_h is fitted on training rows only and then frozen, the first of several strictly causal transforms (Section 7). Because the price almost always drifts past the (tiny) spread at long horizons, the neutral class shrinks sharply with h; the majority-class baseline therefore differs by horizon, a point we return to when reading the accuracy numbers in Section 8.

Distribution and execution targets. Beyond the point return and its sign, the multi-task model (Section 6) predicts a set of return quantiles $\hat{Q}_\tau$, $\tau \in \{0.05, 0.50, 0.95\}$, and, per side, two execution-relevant quantities: the *markout* $M_{t,h}^{\mathrm{side}}$ (the post-fill value of a passive quote left at the touch) and the *adverse-selection* cost $A_{t,h}^{\mathrm{side}} = \max\big(0, -M_{t,h}^{\mathrm{side}}\big)$. These are defined fully in Section 4.7; here it suffices that all targets share a single horizon-indexed trunk and are masked to the rows on which they are available.

3.3 Temporal distribution shift

The training, validation, and test sets are disjoint, chronologically ordered months — not a random partition. Writing $\mathcal{P}_{\text{train}}$ and $\mathcal{P}_{\text{test}}$ for the joint law of features and target on the training and held-out months,

$$\mathcal{P}_{\text{train}}(\mathbf{X}_t,\, y_t) \;\neq\; \mathcal{P}_{\text{test}}(\mathbf{X}_t,\, y_t) \tag{5}$$

in general: volatility, spread, and liquidity regimes differ from month to month. The goal is therefore not i.i.d. generalisation but *robustness across held-out monthly regimes*. Every fitted transform, the direction threshold ε_h, the regime bucket edges, and the feature scaler, is estimated on past (training) months only and applied unchanged to the future, so no statistic is ever computed with knowledge of the test month. The expanding walk-forward protocol that operationalises this is given in Section 7.

3.4 Decision value

Predictive accuracy on the targets of Section 3.2 is necessary but not sufficient: a forecast is only useful if it improves a decision once costs are charged. We judge it against two functionals, both evaluated only on held-out months and detailed in Section 9.

Taker objective. A directional signal is converted to marketable orders by a threshold rule, buy if $\hat{r}_{t,h} > \theta$ and sell if $\hat{r}_{t,h} < -\theta$, with each trade crossing the spread and paying a taker fee. The objective is the total net profit-and-loss $\Pi(\theta) = \sum_t \mathrm{PnL}_t^{\text{net}}(\theta)$ over the test month, where the threshold θ is selected on validation and frozen. This is a sanity check on whether any point-return signal is large enough to overcome execution cost.

Passive market-making objective. The more demanding test asks whether forecasts improve *liquidity provision*. A policy quotes passively at the touch (or deeper), accrues inventory x_t, and is rewarded per decision by

$$R_t = \Delta W_t - \lambda_{\text{inv}} x_t^2 - \lambda_{\text{turn}} |\Delta x_t| - \lambda_{\text{dd}} D_t - \lambda_{\text{adv}} A_t, \tag{6}$$

where ΔW_t is the change in marked wealth, A_t the realised adverse-selection cost of any fill, D_t a drawdown term, and the $\lambda_\bullet$ penalise inventory, turnover and drawdown. The fill model, the wealth accounting, and the realised adverse-selection cost are specified in Section 9.2.

These two functionals make the report's organising distinction operational: Section 8 measures predictive skill on the targets of Section 3.2, whereas Section 9 measures decision value through the taker objective and the market-making reward (6). A model may do well on the first and still fail the second, and as the results will show, that is exactly what happens.

4 Data, Microstructure, Features and Labels

The raw input is the Tardis `book_snapshot_5` feed for BTCUSDT on Binance futures. I reconstruct the top five book levels from these snapshots, derive the microstructure quantities the models consume, and build the features, regime descriptors, and labels on top of them, each computed causally. Quantities are defined once, in the order the pipeline computes them: raw snapshots, reconstruction, microstructure features, the per-day causal rules, regimes, labels, and the execution-relevant targets, closing with the data's observability limits.

4.1 The raw data

The study consumes Tardis `book_snapshot_5` archives for the Binance USDT-margined-futures contract BTCUSDT: one gzipped CSV per trading day, each row a full top-5 snapshot of both sides of the book with microsecond exchange and local timestamps,

```
exchange, symbol, timestamp, local_timestamp,
asks[0..4].{price, amount}, bids[0..4].{price, amount}.
```

The venue writes a row on *every* change to the top of the book, so the feed is large and highly redundant: consecutive rows are frequently identical except for a single size. The six-day development campaign alone totals 7,889,679 raw top-5 snapshots.

Because the feed is so redundant, we subsample in *event time*, retaining every tenth snapshot (`event_stride` $= 10$; Section 4.2). This reduces the six development days from 7,889,679 to 788,970 book rows while preserving the event-

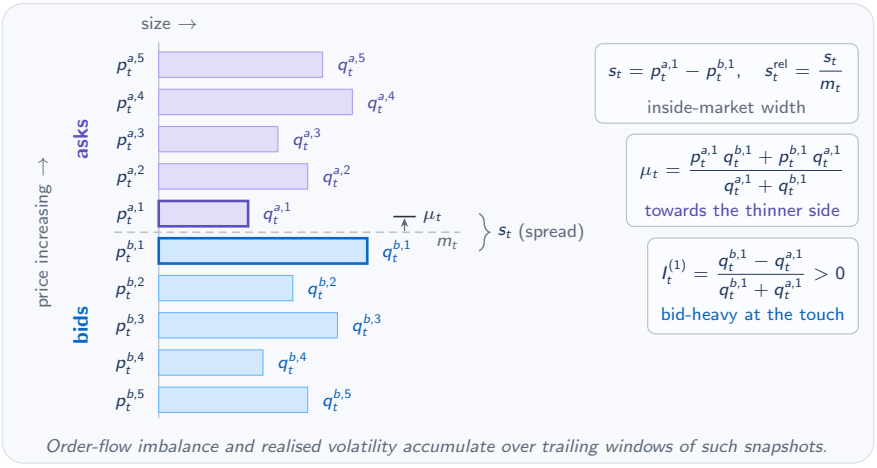


Figure 2: Top-5 limit order book snapshot with the mid, spread, microprice, and level-1 imbalance annotated.

driven structure of the data. The headline walk-forward study (Section 7) extends this calendar to twelve first-of-month days spanning 2025-07 to 2026-06, which yield 1,612,921 feature–label rows after striding.[1]

4.2 Order-book reconstruction and event-time sampling

Each Tardis row is already a complete top-5 snapshot, so reconstruction is conceptually simple: validate the row, derive the reference quantities below, flag anomalies (a crossed book, missing levels, or a detected update gap), and emit one canonical book observation. Recall the mid-price m_t from (2); the spread, relative spread, and microprice are

$$s_t = p_t^{a,1} - p_t^{b,1}, \qquad s_t^{\text{rel}} = \frac{s_t}{m_t}, \qquad \mu_t = \frac{p_t^{a,1}\, q_t^{b,1} + p_t^{b,1}\, q_t^{a,1}}{q_t^{a,1} + q_t^{b,1}}. \tag{7}$$

The microprice μ_t is a size-weighted mid that leans toward the side with *less* displayed size, and is therefore a simple one-step predictor of where the mid will move. Figure 2 illustrates the data object and these quantities.

Vectorised reconstruction. Because every snapshot is a full, canonically shaped top-5 book, reconstruction admits a fully vectorised fast path: an entire day's events are reshaped into an $(n_{\text{snap}} \times 2K)$ array and every derived field is computed in NumPy with no Python per-group loop. On the January day this reduced reconstruction from 1272 s to 4 s, a $\sim$300$\times$ speedup, and was verified column-for-column identical to the general engine, including the quality flags. Engineering detail is in Section 11.

Event-time striding is applied *once*, here at the book stage, so that the features, the taker backtest, and the market-making simulator all share a single sampled event stream and the horizon semantics stay coherent across the pipeline. After striding, the six development days reconstruct in roughly 50 s in total.

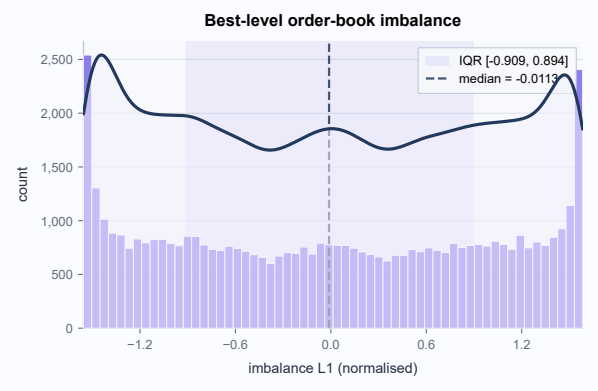
4.3 Microstructure features

From the reconstructed book we compute a compact, causal feature set. Every feature at event t uses only information available at or before t (trailing rolling windows and lagged differences only); the per-day, leakage-safe construction is detailed in Section 4.4 below. The features fall into four groups.

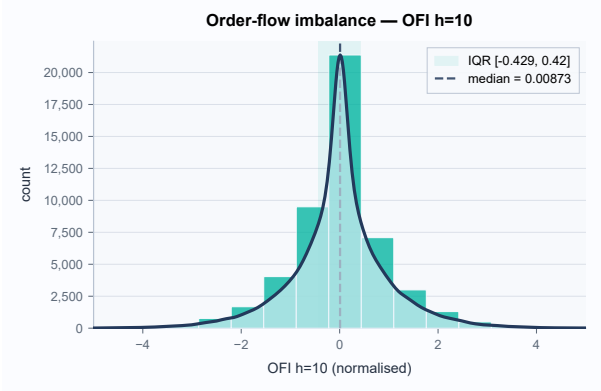
Imbalance. The signed share of displayed size on the bid side, at the touch and across depth, is

$$I_t^{(1)} = \frac{q_t^{b,1} - q_t^{a,1}}{q_t^{b,1} + q_t^{a,1}}, \qquad I_t^{(K)} = \frac{\sum_{k=1}^{K} w_k\, q_t^{b,k} - \sum_{k=1}^{K} w_k\, q_t^{a,k}}{\sum_{k=1}^{K} w_k\, q_t^{b,k} + \sum_{k=1}^{K} w_k\, q_t^{a,k}}, \qquad w_k = \tfrac{1}{k}. \tag{8}$$

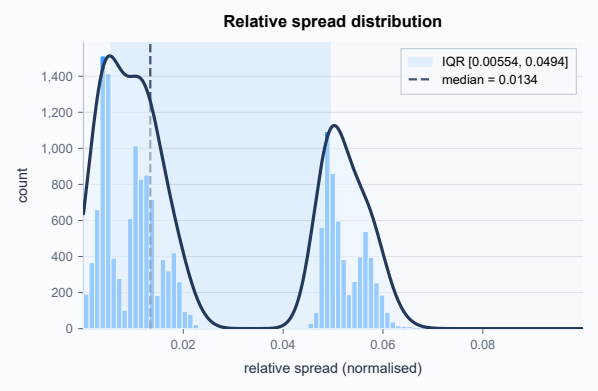
[1] Counts are reported for the twelve-month expanding walk-forward evaluation (Section 7); the label-availability arithmetic in Section 4.6 reproduces this total exactly. Striding is the only subsampling applied; no rows are dropped except where a forward label is unavailable.



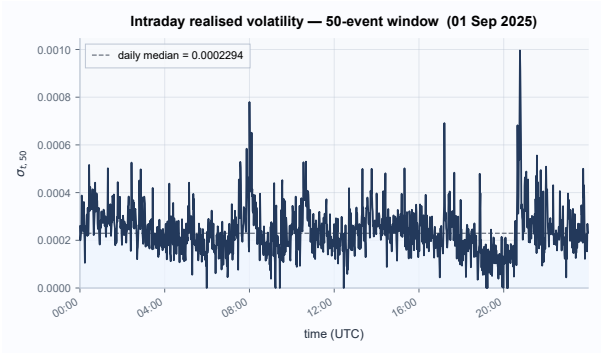
(a) Level-1 imbalance $I^{(1)}$



(b) Order-flow imbalance



(c) Spread



(d) Intraday realised volatility $\sigma_{t,50}$

Figure 3: Empirical distributions of level-1 imbalance, order-flow imbalance, spread, and intraday volatility $\sigma_{t,50}$ over a representative day.

Both lie in $[-1, 1]$ and are positive when bid-side size dominates (buy pressure); the depth-weighted variant $I^{(K)}$ down-weights levels further from the touch by $w_k = 1/k$.

Order-flow imbalance. The signed change in best-level depth attributable to order flow rather than to a price move is accumulated over a trailing window of length L:

$$e_i = \mathbf{1}\{p_i^{b,1} \geq p_{i-1}^{b,1}\}\, q_i^{b,1} - \mathbf{1}\{p_i^{b,1} \leq p_{i-1}^{b,1}\}\, q_{i-1}^{b,1} - \mathbf{1}\{p_i^{a,1} \leq p_{i-1}^{a,1}\}\, q_i^{a,1} + \mathbf{1}\{p_i^{a,1} \geq p_{i-1}^{a,1}\}\, q_{i-1}^{a,1}, \qquad \mathrm{OFI}_{t,L} = \sum_{i=t-L+1}^{t} e_i, \tag{9}$$

with $e_0 = 0$. A positive OFI indicates net bid-side pressure. We use windows $L \in \{10, 50, 200\}$ events.

Lagged returns and realised volatility. Trend and scale are summarised by

$$\mathrm{ret}_L(t) = \frac{m_t - m_{t-L}}{m_{t-L}}, \qquad \sigma_{t,W} = \sqrt{\sum_{i=t-W+1}^{t} \delta_i^2}, \qquad \delta_i = \ln \frac{m_i}{m_{i-1}}, \tag{10}$$

with return lags $L \in \{10, 50, 200\}$ and realised-volatility windows $W \in \{50, 200, 1000\}$. The raw top-5 prices and sizes are also carried through as features, so the model sees both the engineered summaries and the underlying book.

Figure 3 shows the empirical distributions of three core features on the study data, and Table 1 gives summary statistics for a representative subset.

Table 1: Summary statistics for a representative feature subset (twelve-day study set); full catalogue in Appendix A. Spread and prices in USDT.

Feature	Mean	Std	Median	Max
Relative spread	1.07×10^{-6}	3.73×10^{-6}	8.69×10^{-7}	1.19×10^{-3}
Spread (USDT)	0.123	0.427	0.100	136.1
Imbalance $I^{(1)}$	-0.005	0.673	-0.006	1.000
Imbalance $I^{(K)}$	-0.004	0.670	-0.004	1.000
OFI_{50}	0.065	69.6	-0.252	527.2

The statistics make the microstructure concrete. The spread sits at a single tick (0.10 USDT, the contract's tick size) for the overwhelming majority of events (its median is exactly one tick), yet its maximum of 136 USDT during stressed moments shows how violently liquidity can evaporate. In relative terms the spread is sub-basis-point ($\sim 10^{-6}$), so the predictive problem is genuinely about *micro*-moves. Imbalance is centred near zero but U-shaped (standard deviation 0.67, mass near ± 1), and order-flow imbalance is symmetric with heavy tails that widen with the accumulation window. These heavy tails and the evident regime structure are what motivate the regime descriptors of Section 4.5; together they are the raw materials from which the regimes and labels below are built.

4.4 Causal, per-day feature computation

Every feature at event t is a function of the book at or before t only: trailing rolling windows and lagged differences, never a forward shift. Because each first-of-month day is an independent episode (Section 3.1), features are computed *one day at a time* and concatenated, so no rolling window can reach across a day boundary:

```
day_feats  = [compute_features(book_day, cfg) for book_day in days]
feature_df = concat(day_feats).sort_values("timestamp_exchange_ns")
feature_df["month_index"] = feature_df["monthly_date"].map(date_index)
```

Alongside the microstructure features, `compute_features` stamps each row with its `monthly_date`, `month_index`, an `is_first_day_of_month` flag, a coarse time-of-day bucket, and the continuous regime descriptors of Section 4.5. The first W rows of each day, for which a trailing window of length W is not yet full, carry missing descriptor values ($\langle \mathrm{NA} \rangle$) rather than being back-filled, which would not be causal. The leakage-safety of the windowing is enforced again at the dataset stage and verified by the test suite (Sections 5 and 11).

4.5 Regime descriptors and causal bucketing

To let the models and policies condition on prevailing market conditions, each event is tagged with three continuous regime descriptors (volatility, spread, and depth) accumulated causally over a trailing window of $W = 1000$ events:

$$\rho_t^{\mathrm{vol}} = \sqrt{\sum_{i=t-W+1}^{t} \delta_i^2}, \qquad \rho_t^{\mathrm{spr}} = s_t^{\mathrm{rel}}, \qquad \rho_t^{\mathrm{dep}} = \sum_{k=1}^{K} q_t^{b,k} + \sum_{k=1}^{K} q_t^{a,k}, \tag{11}$$

with $\delta_i = \ln(m_i / m_{i-1})$ the one-step log return. Each descriptor is then bucketed into three ordinal regimes by thresholds fitted on the training months only. For volatility,

$$\mathrm{vol\_regime}_t = \begin{cases} \text{low}, & \rho_t^{\mathrm{vol}} \le q_{0.33}, \\ \text{medium}, & q_{0.33} < \rho_t^{\mathrm{vol}} \le q_{0.67}, \\ \text{high}, & \rho_t^{\mathrm{vol}} > q_{0.67}, \end{cases} \tag{12}$$

where $q_{0.33}$ and $q_{0.67}$ are the 33rd and 67th percentiles of ρ^{vol} over the canonical training months; spread and depth are bucketed analogously into $\{\text{tight}, \text{normal}, \text{wide}\}$ and $\{\text{thin}, \text{normal}, \text{deep}\}$. The bucket edges are frozen and reused unchanged on validation and test, so the regime labelling is strictly causal. A coarse four-hour UTC time-of-day bucket is computed directly (it requires no fitting). The four categoricals (volatility, spread, liquidity, and time-of-day regime) are mapped to a fixed-width one-hot context vector $\mathbf{c}_t$ (an unrecognised or missing level maps to all-zeros), which feeds both the deep model's context branch (Section 6.4) and the market-making policy state (Section 9).

Where the edges are fitted. In the walk-forward design the "canonical training months" are the *earliest* fold's training months. Fitting every transform, the direction threshold ε_h of Section 4.6 and the regime edges here, on the earliest data and applying them forward guarantees that a frozen *past* statistic is applied to the *future*, never the reverse.

4.6 Labels and their class structure

The forward return $r_{t,h}$ (3) and the thresholded direction $y^{\text{dir}}_{t,h}$ (4) were defined in Section 3.2. Two points of construction matter here. First, like the features, labels are computed *per day*, so the last h rows of each day, whose horizon would fall into the next day, carry no label and are excluded. Second, the threshold $\varepsilon_h = \alpha \,\text{median}_{\text{train}}(s^{\text{rel}}_t)$ is fitted on the canonical training months only and frozen, so the class definition is stable across all held-out months.

For the probabilistic head, the model predicts return quantiles $\hat{Q}_\tau$, $\tau \in \{0.05, 0.50, 0.95\}$, trained with the pinball (quantile) loss and assessed by empirical interval coverage:

$$\mathcal{L}_\tau(r, \hat{Q}_\tau) = \max\{\tau(r - \hat{Q}_\tau),\ (\tau - 1)(r - \hat{Q}_\tau)\}, \qquad \widehat{C}_{90} = \frac{1}{N}\sum_t \mathbf{1}\{\hat{Q}_{0.05} \le r_t \le \hat{Q}_{0.95}\}, \tag{13}$$

with a nominal target of $\widehat{C}_{90} = 0.90$ for the central 90% interval. Table 2 gives the label availability and class balance by horizon on the twelve-day study set.

Table 2: Class balance and label availability by horizon; the majority-class baseline is the most-frequent-class prediction accuracy.

Horizon	Available	Up	Neutral	Down	Majority acc.
$h = 10$	1,612,801	596,990	411,995	603,816	0.374
$h = 50$	1,612,321	768,548	68,255	775,518	0.481
$h = 200$	1,610,521	799,381	6,203	804,937	0.500

The collapse of the neutral band is the single most important thing to keep in mind when reading the predictive results. At h=10 the three classes are comparably populated (roughly 37% up, 26% neutral, 37% down), so the majority-class baseline is only about 0.374. By h=50 the price has almost always moved past the (sub-basis-point) spread, the neutral class falls to about 4%, and the problem is nearly balanced binary with a baseline near 0.48; by h=200 the neutral class is negligible and the baseline is essentially the coin-flip line at 0.50. Consequently a raw accuracy of, say, 0.53 is unimpressive at h=10 but a genuine edge at h=200, and all accuracy figures in Section 8 are read against the per-horizon (and per-month) majority baseline rather than a fixed 0.5.

4.7 Markout and adverse-selection targets

The execution-aware heads are trained on targets that proxy the value of a *passive* quote left at the touch. Reconstructing the best bid and ask from the mid and spread, $p^{b,1}_t = m_t - s_t/2$ and $p^{a,1}_t = m_t + s_t/2$, the markout and adverse-selection targets at horizon h are

$$M^{\text{bid}}_{t,h} = m_{t+h} - p^{b,1}_t, \qquad M^{\text{ask}}_{t,h} = p^{a,1}_t - m_{t+h}, \qquad A^{\text{side}}_{t,h} = \max(0,\ -M^{\text{side}}_{t,h}). \tag{14}$$

The markout M^{side} is the post-fill mark-to-mid of a passive quote on that side h events later, and the adverse-selection cost A^{side} is its loss-making part: the amount by which the mid moved *through* the quote, which is exactly what a liquidity provider is run over by. These are simple *proxies*: they assume a fill at the touch and mark against the mid; they do not model whether and when a resting order would actually have traded. That gap between proxy target and realised fill is taken up by the queue-aware fill model of Section 9, and it is why the execution-aware heads are evaluated as forecasts (Section 8.4) separately from the decision-value tests that consume them (Section 9).

4.8 Observability constraints

It is worth stating the data's epistemic limits at the outset, because they bound what the execution simulator of Section 9 can legitimately claim. Top-5 snapshots reveal *displayed* depth at the inner five levels but not true queue position; there is no market-by-order feed, and no separate trade or aggressor-flow stream. Event-time sampling changes the meaning of a horizon (Section 4.1), and first-of-month sampling is a deliberate regime-snapshot design rather than continuous market coverage. Table 3 classifies the quantities the study relies on.

Table 3: Data observability: what can be seen, reconstructed, approximated, or is unavailable. The approximated quantities bound what the execution simulator can claim.

Quantity	Observed	Reconstructed	Approximated	Unavailable
Top-5 prices and sizes	✓			
Mid, spread, microprice		✓		
Imbalance, order-flow imbalance		✓		
Own-order queue position			✓	(true value)
Passive-order fills			✓	(true fills)
Full book depth (beyond 5)				✓
Trade prints / aggressor flow				✓

These constraints are not incidental. They are the reason the report treats queue position and passive fills as explicitly *approximated* quantities (Section 9), and reports its decision-value conclusions under that caveat rather than presenting the simulator as ground truth.

5 System Architecture and Leakage Discipline

When the signal is weak, a single accidental look-ahead can manufacture an edge that does not exist, so the result is only as good as the pipeline that produces it. The platform reduces leakage risk by separating stages, writing explicit artefacts at each boundary, and testing the causal invariants at those boundaries. This section covers the staged dataflow (Section 5.1), the module layout (Section 5.2), the data contracts that let stages and runs interoperate (Section 5.3), and the causal invariants that the pipeline and its test suite enforce (Section 5.4).

5.1 End-to-end dataflow

The pipeline is a sequence of pure stages. Each stage reads explicit inputs, writes explicit outputs (Parquet for data, small YAML index files for metadata), validates what it wrote against the schema it promises, and holds *no* hidden global state. The practical consequences are that every stage is independently testable and replaceable, that a run can be resumed from any completed stage, and that the artefacts of one run can be compared against another without re-execution. Figure 4 shows the full path from raw archives to the final report and verdict.

Two engineering choices in the data layer are worth singling out because they protect correctness directly. Ingestion writes files *atomically* (to a temporary path, then an atomic rename) and records a `sha256` of every raw file in a manifest, so a partially written or silently altered input can never be mistaken for a good one. Normalisation never silently drops malformed rows: prices ≤ 0 or negative sizes are nulled *and flagged*, duplicate-id rows are flagged, and a monotone `event_id` is assigned per day. These quality flags travel with the data all the way to the book table, so anomalies remain visible to every downstream consumer.

5.2 Repository and module organisation

The implementation mirrors the dataflow: one package per stage, each exposing a small, typed interface. The trimmed layout is

```
src/lob_forecasting/
  config/        schema.py loader.py          # typed config + hard invariants
  ingestion/     manifest.py sources.py ingest.py
  normalisation/ event_table.py adapters.py normalise.py
  orderbook/     book_state.py reconstruct.py # vectorised fast path + striding
  features/      compute.py regimes.py build.py
  labels/        compute.py quantiles.py markout.py build.py
  datasets/      splits.py monthly_splits.py scaler.py build.py
  models/        registry.py baselines.py linear.py gbm.py tcn.py
                 deep/tcn_exec_multitask.py    # multi-task execution model
  training/      orchestrate.py
  evaluation/    metrics.py robustness.py distributional.py bootstrap.py
  backtesting/   engine.py run.py  market_making/   # fills, accounting, policies
  diagnostics/   tables.py figures.py monthly_report.py
```

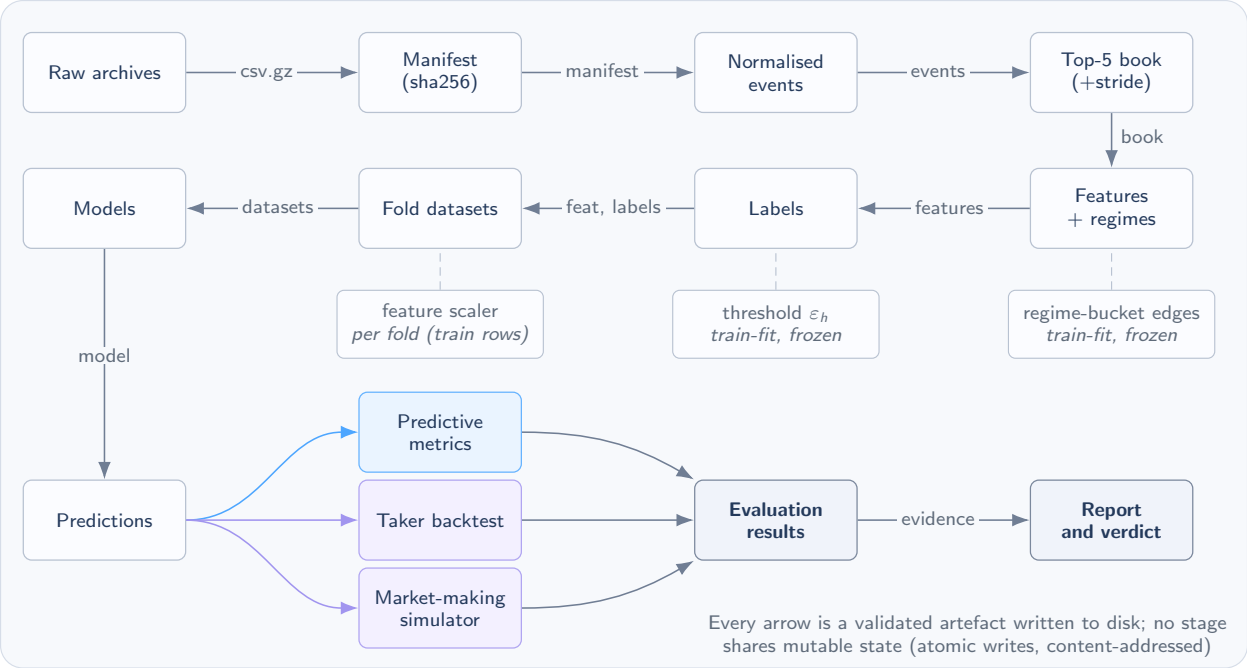


Figure 4: Research pipeline from raw archives to the final report; each arrow is a validated artefact and stages share no mutable state.

A typed configuration object (validated once, at load time; Section 5.4) is threaded through every stage, so no stage re-parses YAML or invents its own defaults. A decorator registry maps model names to classes and gates which models a configuration may request, so adding a model is a local change that the rest of the pipeline need not know about. The full module-dependency diagram and the complete configuration schema are deferred to Appendix B.

5.3 Data contracts

Stages communicate only through stable, documented column schemas, the *data contracts*, so a later stage never needs to know how an earlier one was implemented. Table 4 summarises the four central artefacts.

Table 4: The four core data contracts: schema and consumers for each pipeline stage.

Artefact	Grain	Key fields	Train-fitted?	Consumed by
`events`	one event	`event_id` (resets per day), timestamp, side, price, qty, flags	no	reconstruction
`book_top5`	one book obs. (post-stride)	L1–5 prices/sizes, mid, spread, microprice, flags	no	features, simulator
`features_labels`	one event	features, regime tags, all labels	thresholds & edges (train only)	datasets, training
`predictions/`m	one event × horizon	direction, return, quantiles, markout, adverse	model-dependent	metrics, backtests, MM

The prediction contract deserves emphasis. Every model, whether a trivial baseline, a linear model, or the multi-task TCN, writes the *same* long-format table, one row per event per horizon, with nullable columns for the execution-aware heads (quantiles, markout, adverse) that only some models fill. A new model or an added horizon simply appends rows; the evaluation, backtest, and market-making code never changes. This single schema is also what makes the predictions of one run reusable by another: because each file is self-describing (it carries its `run_id`, `model_name`, split, and timestamps), a later run can re-evaluate or re-backtest an earlier run's frozen predictions without retraining. Every run is stored fully self-contained under its own `run_id`, so independent runs coexist on disk and are compared directly (Section 11).

5.4 Leakage discipline: the invariants

Three classes of leakage threaten a study like this: a feature or label window that reaches across the calendar discontinuity; a fitted transform that has seen the future; and a join that silently mixes rows from different days. The pipeline closes all three.

Calendar discontinuity. Because each monthly day is an independent episode (Section 3.1), any retained window of length L ending at event t with forward horizon h must satisfy

$$\mathrm{day}(t - L + 1) = \mathrm{day}(t) = \mathrm{day}(t + h), \tag{15}$$

i.e. the entire input window *and* the label horizon lie within one day. Features and labels are computed per day (Section 4); sequence windows that would straddle two days are rejected; and market-making inventory is reset to flat at the start of each day (Section 9). Concretely, a sequence window ending at index `end` (`start` $=$ `end` $- L + 1$) is admitted only if three $O(1)$ prefix-sum tests pass:

```
pref[end+1]       - pref[start]       == 0     # no invalid/NA feature row in the window
split_pref[end+1]- split_pref[start] == L     # window lies wholly inside this split
day_code[start]   == day_code[end]            #   (not the embargo) and within one day
```

In words: an admitted window sits entirely inside one split and one day, clear of the embargo gap, whereas a window that crosses the day boundary is rejected. The embargo (here 200 events, at least the longest horizon) removes rows whose label horizon would otherwise peer into the validation or test split.

Causal fitted transforms. Every statistic that touches the data is estimated on past data and frozen. The direction threshold ε_h and the regime bucket edges are fitted on the earliest fold's training months (Section 4); the feature scaler is fitted per fold on *that fold's* training rows only, standardising each feature by its training mean and standard deviation, $z = (x - \hat{\mu}_{\text{train}})/\hat{\sigma}_{\text{train}}$, and then applied unchanged to validation and test. No normalisation, threshold, or bucket edge is ever computed with knowledge of a held-out month.

Join on time, never on `event_id`. Because `event_id` is monotone only *within* a day and resets across days, joining metrics or context back onto predictions by `event_id` would silently align rows from different days. All such joins therefore key on `(venue, symbol, timestamp_exchange_ns)` instead.

Fail-fast configuration. Finally, many leakage and scope errors are caught before any computation runs: the configuration loader validates the run once into a typed object and *refuses* configurations that violate the study's invariants. Among the checks:

```
symbols != ["BTCUSDT"]                          -> error
top_k   != 5                                    -> error
sampling.mode != "event_time"                   -> error
embargo_events < max(horizons_events)           -> error   # embargo must cover the horizon
market_making.enabled and not markout_targets   -> error
contextual_bandit_mm requested, no validation   -> error   # no fold to select on
a monthly date that is not the first of a month -> error
event_stride < 1                                -> error
```

Together these invariants close off the main ways one could accidentally inflate performance, and the test suite (Section 11) asserts each of them. This is what lets the "no decision value" outcome of Section 9 be read as a property of the problem rather than the residue of a leak.

6 Models

The roster spans trivial baselines, linear and gradient-boosted tabular models, a compact direction-only temporal convolutional network (TCN), and an execution-aware multi-task TCN with a latent state-space context. The baselines are not filler: they define the bar a deep model must clear, and as Section 8 shows, that bar is high. Every model is trained *per horizon* and writes the common long-format prediction contract of Section 5.3, so all of them are evaluated by exactly the same downstream code. Table 5 summarises the roster.

6.1 Model interface and registry

All models implement one abstract interface — `fit`, `predict`, `save`, `load`, `hyperparameters`, and a class flag `requires_sequences` — and register themselves by name in a decorator registry that gates which models a configuration may request. Tabular classifiers wrap an `sklearn` estimator and always return an $(n, 3)$ probability over the classes $\{-1, 0, +1\}$; sequence models consume causal windows. To keep memory bounded, the sequence loader is *lazy*: it caches the per-symbol scaled feature matrices once and stores each window only as a `(matrix, start)` index plus its window-end context vector $\mathbf{c}_t$, materialising the (batch, L, F) tensor only when a batch is requested. The save/load round-trip (state dict, hyperparameters, context dimension) is tested to reproduce predictions bit-for-bit.

Table 5: Model roster. Tabular models fit one estimator per horizon; sequence models share a trunk with per-horizon heads. Training details in Section 7.

Model	Type / input	Key hyperparameters	Selection
`no_change`	anchor (none)	predicts return 0 / neutral	—
`imbalance_rule`	rule (raw $I^{(1)}$)	threshold $\gamma \in \text{linspace}(0, 0.9, 19)$	best mean val. accuracy
`logistic_regression`	linear 3-class (tabular)	$C = 1.0$, max_iter 1000	—
`ridge_regression`	linear return (tabular)	$\alpha = 1.0$	—
`lightgbm`	GBDT 3-class (tabular)	31 leaves, lr 0.05, 300 trees, subsample 0.8	—
`tcn_small`	causal TCN, direction (seq.)	ch. 16, kernel 5, 4 layers, dropout 0.1	composite val. score (§8)
`tcn_exec_multitask`	multi-task TCN (seq. + ctx.)	ch. 32, kernel 5, 5 layers, dropout 0.1	composite val. score (§8)

6.2 Baselines

Four baselines bracket the achievable skill. `no_change` is the trivial anchor: it predicts a zero return and the neutral class, so its accuracy is exactly the majority-class rate of Section 4.6 and exposes how the bar moves with horizon. `imbalance_rule` is an interpretable microstructure rule that trades on the sign of the level-1 imbalance once $|I^{(1)}|$ exceeds a threshold γ selected on validation accuracy — a sanity check on how much of the signal is just order-book pressure. `logistic_regression` (3-class) and `ridge_regression` (point return) establish the *linear* skill ceiling on the full tabular feature set, and `lightgbm` adds a non-linear gradient-boosted reference. A deep sequence model that cannot beat these is not learning anything the features do not already expose linearly.

6.3 The direction-only TCN

`tcn_small` is a compact causal TCN that predicts only the direction class, one softmax head per horizon. It stacks n dilated causal convolutions with kernel size k and exponentially growing dilations $1, 2, 4, \ldots, 2^{n-1}$, so that the output at event t depends only on inputs at or before t. The receptive field of such a stack is

$$R = 1 + (k - 1)(2^n - 1). \tag{16}$$

With kernel $k = 5$ and $n = 5$ layers the larger multi-task encoder below reaches $R = 125 \geq L = 100$, spanning the whole input window; the smaller direction-only `tcn_small` ($k = 5$, $n = 4$) has $R = 61$ and so concentrates on the most recent events. The dilation stack and its causal cone are shown as an inset of the architecture figure (Figure 5). `tcn_small` is, on the evidence of Section 8, the project's best direction model at the shortest horizon; its small size and regularisation (dropout, weight decay) are exactly what made it generalise across held-out months rather than overfit (Section 10).

6.4 The execution-aware multi-task TCN

The centrepiece of the study is `tcn_exec_multitask`, designed to emit *exactly* the quantities a passive-quoting policy consumes. A shared causal TCN encoder maps the input window to a sequence of hidden states; these are pooled over time, fused with the regime/context vector, and read out by per-horizon multi-task heads. Figure 5 shows the full forward graph.

Temporal pooling. The default pooling is a learned *gated* average over the encoder hidden states $h_{t-L+1}, \ldots, h_t$:

$$g_i = \sigma(w_g^\top h_i + b_g), \qquad \alpha_i = \frac{g_i}{\sum_j g_j + \epsilon}, \qquad \mathbf{z}_t = \sum_i \alpha_i h_i, \tag{17}$$

which lets the model weight informative events; last-step and single-head attention pooling are available as alternatives (compared in Section 10).

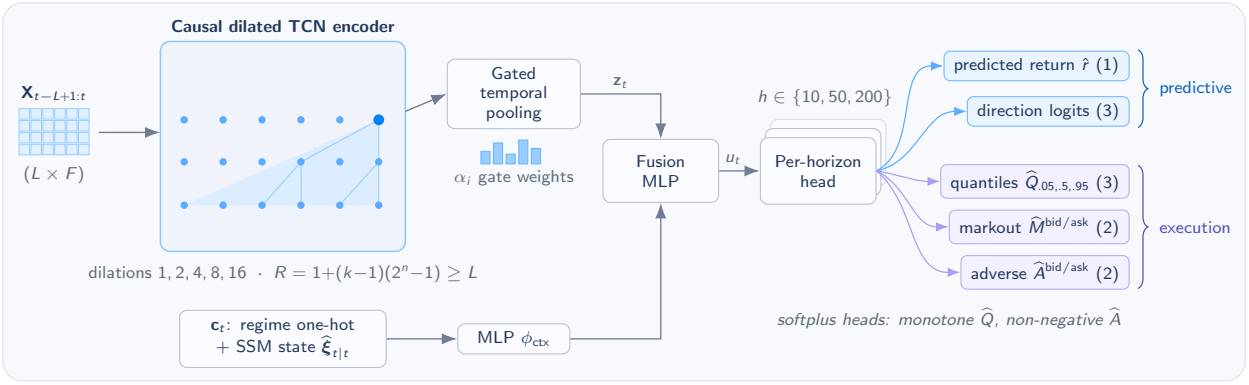


Figure 5: The multi-task TCN: shared encoder feeding per-horizon heads for direction, return quantiles, and markout and adverse-selection estimates.

Context fusion. The pooled representation is fused with an embedding of the regime/context one-hot $\mathbf{c}_t$ (Section 4.5) through two small MLPs,

$$u_t = \phi_{\text{fusion}}\big([\mathbf{z}_t,\ \phi_{\text{ctx}}(\mathbf{c}_t)]\big), \tag{18}$$

and a per-horizon head g_h maps u_t to eleven outputs: a point return (1), direction logits (3), three raw quantile increments, and bid/ask markout (2) and raw adverse (2).

Structurally valid heads. Two outputs are constrained by construction. The return quantiles are built by cumulative softplus increments, which guarantees monotonicity $\hat{Q}_{0.05} \leq \hat{Q}_{0.50} \leq \hat{Q}_{0.95}$, and the adverse-selection head is passed through a softplus so it is non-negative:

$$\hat{Q}_{0.05} = u_1, \quad \hat{Q}_{0.50} = \hat{Q}_{0.05} + \text{softplus}(u_2), \quad \hat{Q}_{0.95} = \hat{Q}_{0.50} + \text{softplus}(u_3), \qquad \widehat{A}^{\text{side}} = \text{softplus}(v^{\text{side}}) \geq 0. \tag{19}$$

Multi-task objective. All heads are trained jointly under a config-weighted loss, averaged over horizons ($\lambda_h = 1/|\mathcal{H}|$) and masked to the rows on which each label is available:

$$\mathcal{L}(\theta, \phi) = \sum_{h \in \mathcal{H}} \lambda_h \Big[\omega_r \text{Huber}(r_{t,h}, \hat{r}_{t,h}) + \omega_d \text{CE}(y^{\text{dir}}_{t,h}, \hat{p}_{t,h}) + \omega_q \sum_{\tau \in \{.05,.5,.95\}} \mathcal{L}_\tau(r_{t,h}, \hat{Q}_{\tau,t,h}) + \omega_m \mathcal{L}^M_{t,h} + \omega_a \mathcal{L}^A_{t,h}\Big], \tag{20}$$

where $\mathcal{L}_\tau$ is the pinball loss (13) and $\mathcal{L}^M$, $\mathcal{L}^A$ are Huber losses on the markout and adverse targets (14). The objective weights $(\omega_r, \omega_d, \omega_q, \omega_m, \omega_a)$ are the principal levers of the study (Section 10). From the trained quantiles the model derives an interval width and a scale-free uncertainty score that the policies use as a quoting signal:

$$\text{width}_{t,h} = \hat{Q}_{0.95} - \hat{Q}_{0.05}, \qquad U_{t,h} = \frac{\hat{Q}_{0.95} - \hat{Q}_{0.05}}{|\hat{Q}_{0.50}| + \epsilon}. \tag{21}$$

Objective over size. The multi-task network is small (32 channels, five layers). Its value is not capacity but *interface*: it emits, per horizon, precisely the quantities an execution policy needs (a sign, a calibrated interval, and a markout and adverse estimate per side), so the decision layer of Section 9 can consume the model's outputs directly rather than re-deriving risk from a bare point forecast.

6.5 Return-head ablation variants

A recurring question (Q4) is whether the noisy point-return target *harms* the shared representation. To answer it cleanly the network is wired (v2.0) so that the point-return head is a *separate* linear map $\hat{r}_{t,h} = g^r_{h,\phi}(\mathbf{z}_t)$ whose gradient can be cut from the encoder without touching the direction, quantile, markout, or adverse heads. Two variants carry the ablation reported later: `tcn_exec_base`, which keeps the return head at loss weight 0.10 with its gradient flowing into the encoder, and `tcn_exec_ret0`, which switches the head off entirely. A third *detached* variant keeps the head but stops its gradient at the encoder, so the return loss updates only its own parameters,

$$\nabla_\theta \mathcal{L}_r = 0, \qquad \nabla_\phi \mathcal{L}_r \neq 0, \tag{22}$$

and a fourth replaces the neural head with a ridge sidecar, merging `ridge_regression`'s prediction onto the neural outputs after inference (joining on `(venue, symbol, split, timestamp, horizon)`, never `event_id`; see Section 5.4). The walk-forward evaluation runs `tcn_exec_base`, `tcn_exec_ret0`, and the SSM-augmented `tcn_exec_ret0_ssm`; the detached and ridge-sidecar variants are implemented and unit-tested, but running them at scale was left to future work (Section 12.7).

6.6 Latent state-space context

The final modelling component is a small, time-invariant linear-Gaussian state-space model (SSM) that supplies a smoother *causal* estimate of latent market state (drift, liquidity stress, volatility, order-flow pressure) to the context branch. With a $K = 4$-dimensional latent state $\boldsymbol{\xi}_t$ and a seven-dimensional observation $\mathbf{y}_t$ (the level-1 and depth-weighted imbalance, OFI_{50}, the lag-10 return, realised volatility, relative spread, and depth, standardised on training-month statistics),

$$\boldsymbol{\xi}_{t+1} = A\boldsymbol{\xi}_t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(0, Q); \qquad \mathbf{y}_t = C\boldsymbol{\xi}_t + \mathbf{d} + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(0, R). \tag{23}$$

The emission matrix C is initialised from PCA loadings, A is diagonal in $[0.80, 0.98]$, and Q, R are diagonal; the parameters are refined by a stable EM-style loop (at most 25 iterations) on the training months only, using a bounded subsample because the Kalman filter is an inherently sequential $O(T)$ recursion. The features handed to the network are the causal *filtered* states $\hat{\boldsymbol{\xi}}_{t|t}$ and their variances $\mathrm{diag}(P_{t|t})$, never the smoothed states $\hat{\boldsymbol{\xi}}_{t|T}$, which would peek at the future; the filter prior is *reset at each monthly-day boundary* to respect the calendar discontinuity. These are appended to the context vector $\mathbf{c}_t$ *per variant*, so that an SSM and a no-SSM model are trained and evaluated on otherwise identical data (the paired ablation of Section 10.2), which finds it the single most effective architectural change in the study.

7 Experimental Protocol and Evaluation

The evaluation has two parts: a development split used for model design, and the walk-forward evaluation used for the reported results. This section describes both, together with the metrics and the selection rule. Three properties carry through: testing is out-of-sample by month, model selection touches only the validation split, and the metrics and selection rules are chosen to avoid overstating small improvements over the majority baseline.

7.1 Two evaluation campaigns

The study proceeded in two campaigns, and it is important to keep their roles distinct. A *development campaign* used a single fixed $4/1/1$ monthly split (train January–April, validate May, test June 2026) to design the networks and tune their objective weights, regularisation, and early-stopping rules. With a single test month it is methods development, not a robustness test, and its narrative is reported as ablation context in Section 10 and Appendix D. The *walk-forward evaluation* is the twelve-month expanding walk-forward described below, which evaluates the developed models across *multiple* held-out months and is the basis for every result in Sections 8 and 9. Separating the two ensures that no design decision was made on data later used to report a result.

7.2 Expanding walk-forward protocol

The canonical dataset is one first-of-month BTCUSDT day per month from 2025-07 to 2026-06. The split mode is an expanding window with six training months, one validation month, and one test month, advancing one month at a time (`step_months` $= 1$), which yields the five folds of Figure 6. Training always begins at the earliest month and grows; validation is always the month immediately before the test month; and each fold's test month is the next fold's validation month, so no month is ever used for selection and then for reporting in the same fold. To fit the compute budget the evaluation ran folds 2–4 (test months 2026-04, 2026-05, and 2026-06); folds 0–1, which would add the February and March test months, lie within the same protocol and are left to future work (Section 12.6).

What is frozen, and where. The direction thresholds ε_h (4) and the regime bucket edges (12) are fitted *once*, on the earliest fold's training months, and reused for every fold, so the label and regime definitions are identical across all test months while remaining strictly causal. Feature scalers, by contrast, are re-fitted *per fold* on that fold's training rows, since each fold has a different training window. This is the concrete realisation of the distribution-shift commitment of Section 3.3.

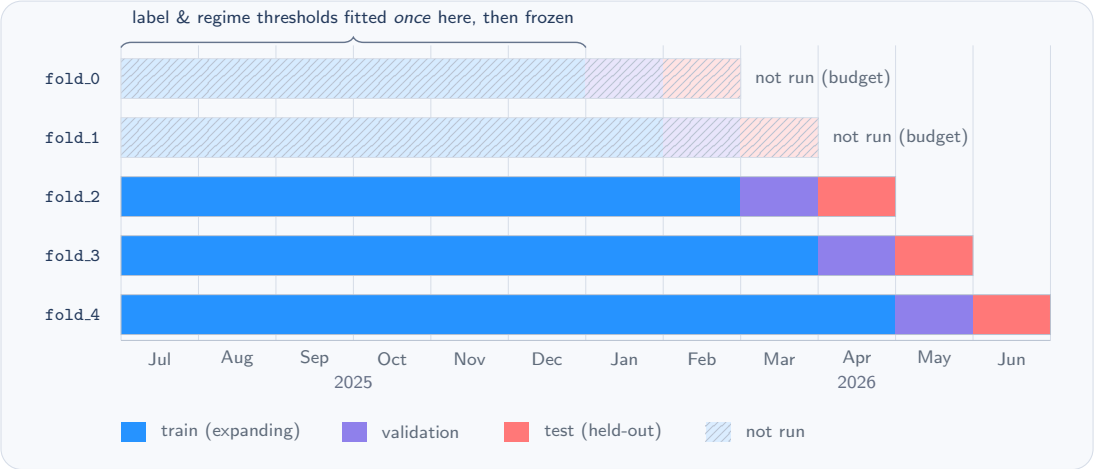


Figure 6: Expanding walk-forward: training grows month by month; test months lie in the future of all fitted data.

7.3 Datasets and split sizes

Within each fold the feature–label table is sliced into train, validation, and test by whole months, sequence windows of length $L = 100$ are built subject to the leakage-safe admission rules of Section 5.4, and an embargo of 200 events (at least the longest horizon) is removed at each split boundary. The training set grows from roughly 1.03M rows (eight months) to 1.28M (ten months), while validation and test are single months of order 10^5 rows each; the per-fold split sizes are tabulated in Appendix C (Table 19).

7.4 Evaluation metrics

Forecasts are assessed with three families of metrics, all computed out-of-sample and joined back to the context table on `(venue, symbol, timestamp_exchange_ns)`, never on `event_id`, which resets per day (Section 5.4).

Classification (direction). Accuracy and *balanced* accuracy (the mean of per-class recalls, which does not reward exploiting the majority class), multinomial cross-entropy, the Brier score, the 3×3 confusion matrix, and the expected calibration error

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|\mathcal{B}_b|}{N} \big|\mathrm{acc}(\mathcal{B}_b) - \mathrm{conf}(\mathcal{B}_b)\big|, \tag{24}$$

where predictions are partitioned into B confidence bins $\mathcal{B}_b$ and acc, conf are the bin accuracy and mean confidence.

Point return. Mean absolute error, root-mean-square error, the rank information coefficient

$$\mathrm{rankIC}_h = \rho_{\mathrm{Spearman}}\big(\hat{r}_{\cdot,h},\, r_{\cdot,h}\big) = \mathrm{corr}\big(\mathrm{rank}\,\hat{r}_{\cdot,h},\, \mathrm{rank}\,r_{\cdot,h}\big), \tag{25}$$

the sign correlation, and the out-of-sample R^2 benchmarked against the *training-period mean return* $\bar{r}_{\mathrm{train},h}$:

$$R^2_{\mathrm{oos},h} = 1 - \frac{\sum_{t\in\mathrm{test}} \big(r_{t,h} - \hat{r}_{t,h}\big)^2}{\sum_{t\in\mathrm{test}} \big(r_{t,h} - \bar{r}_{\mathrm{train},h}\big)^2}. \tag{26}$$

A strongly negative R^2_{oos} means the point forecast is *worse than simply predicting the training mean*, the diagnostic that exposes the multi-task return head's instability in Section 8.

Distributional. The pinball loss per quantile and the empirical 90% interval coverage $\widehat{C}_{90}$ of (13), together with the mean interval width; all three are reported pooled and grouped by month and by regime.

7.5 Model selection and two-phase training

Every model-selection decision is made on the validation month only, and the test month is touched exactly once, to report. For the multi-task network, selecting on direction accuracy alone is unsafe: it can restore a checkpoint at which

the slower quantile, markout, and adverse heads are still under-trained. We therefore select on a *composite* validation score that balances directional skill against calibration and execution-head accuracy. Per horizon,

$$S_h = 0.25\,\mathrm{BA}_h + 0.15\,\mathrm{RIC}_h - 0.20\,\mathrm{QL}_h^{\mathrm{norm}} - 0.15\big|\widehat{C}_{90,h} - 0.90\big| \\ - 0.05\,\mathrm{MW}_h^{\mathrm{norm}} - 0.10\,\mathrm{MAE}_h^{M,\mathrm{norm}} - 0.10\,\mathrm{MAE}_h^{A,\mathrm{norm}}, \qquad S = \frac{1}{|\mathcal{H}|}\sum_{h\in\mathcal{H}} S_h, \tag{27}$$

where BA is balanced direction accuracy, RIC the rank-IC of the median quantile $\hat{Q}_{0.50}$, QL the mean pinball loss, $\widehat{C}_{90}$ the empirical 90% coverage, and MW the interval width. The penalty terms are normalised by per-horizon median-absolute-deviation scale constants fitted on training rows and *capped at* 100, so that a single exploding ratio on an under-trained head cannot dominate checkpoint selection; the terms of any disabled head are dropped and the remaining weights renormalised.

Two-phase training. The multi-task network is trained in two phases. *Phase A* jointly trains all enabled heads with AdamW, checkpointing by the composite score S (with a minimum number of epochs before early stopping can fire, so the slow heads are guaranteed enough training). *Phase B* then freezes the encoder, pooling, context embedding, and fusion MLP, and trains only the calibration heads (quantile, markout, adverse, and the detached return head where enabled) for a few epochs at a lower learning rate with the direction weight set to zero: a dedicated calibration pass that fixed the quantile-coverage fragility seen in development (Section 10.3). Under the walk-forward compute budget each variant ran roughly twelve epochs in total (Phase A up to eight, Phase B four).

7.6 Robustness reporting

Because the goal is robustness across regimes rather than i.i.d. generalisation, results are summarised *across the held-out months*. For each test metric we report its mean and standard deviation over the test months, its best and worst month, the fraction of months in which it is positive, and, for accuracy, the number and fraction of months in which the model beats the per-month majority-class baseline (`n_months_beating_baseline`). Within a month, confidence intervals are obtained by a moving-block bootstrap that resamples contiguous blocks of events (here 300 resamples of 1000-event blocks) so that short-range autocorrelation is preserved, and the $(1-\alpha)$ interval is taken from the percentiles of the resampled statistic. Three test months is a small panel: the cross-month standard deviations are indicative, not tight, and where a statistic has only one month to resample it is reported as not computed. This per-month, beat-the-baseline framing is exactly what distinguishes a one-month fluke from a robust edge in Section 8.

8 Predictive Results

The predictive results are mixed. The best TCN improves on the short-horizon baseline, but the advantage is small and does not hold uniformly across horizons, and the auxiliary heads help calibration more than direction accuracy. The rest of this section reports this in detail against the first three sub-questions — directional signal beyond baselines (Q1), deep models versus the linear and gradient-boosted references (Q2), and whether multi-task learning helps the execution-relevant heads (Q3) — on the three held-out months of the walk-forward evaluation. Throughout, accuracy is read against the *per-horizon* majority-class baseline of Section 4.6 (0.374, 0.481, 0.500 at $h = 10, 50, 200$), not a fixed 0.5, because the collapsing neutral band moves the bar.

8.1 Direction accuracy across held-out months

Table 6 gives mean test accuracy by model and horizon, averaged over the three held-out months, with the fraction of months in which each model beats the per-month majority baseline at $h = 50$.

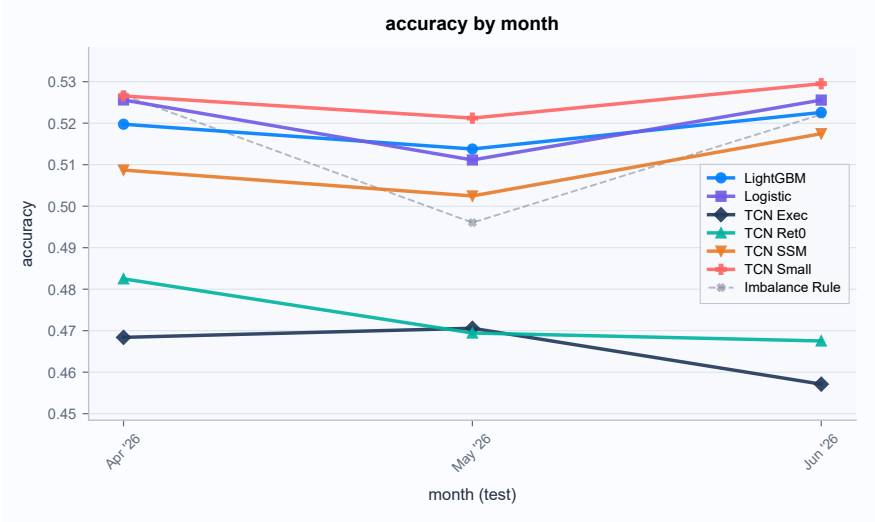


Figure 7: Direction accuracy by held-out month, with per-month majority baseline.

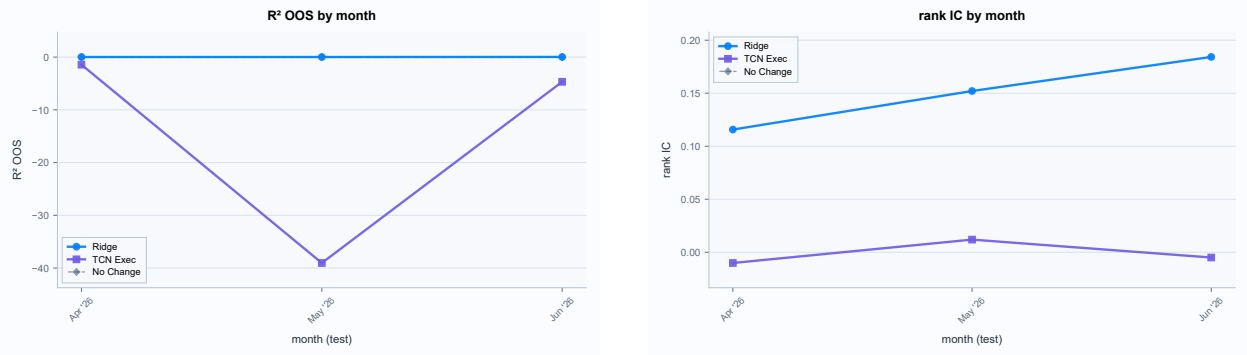
Table 6: Mean test direction accuracy by model and horizon (three held-out months); months beating the per-month majority baseline at h=50. Best value in **bold**.

Model	h=10	h=50	h=200	**Months beat (h50)**
majority-class baseline	*0.374*	*0.481*	*0.500*	—
`no_change`	0.2396	0.0263	0.0031	0/3
`imbalance_rule`	0.4970	**0.5321**	0.5159	3/3
`logistic_regression`	0.5147	0.5310	**0.5165**	3/3
`lightgbm`	0.5286	0.5192	0.5083	3/3
`tcn_small`	**0.5333**	0.5312	0.5128	3/3
`tcn_exec_base`	0.4077	0.4904	0.4979	2/3
`tcn_exec_ret0`	0.4101	0.4969	0.5125	3/3
`tcn_exec_ret0_ssm`	0.5071	0.5192	0.5023	3/3

The headline is `tcn_small`. At the shortest horizon it is the best model (0.5333), clearing the majority baseline by roughly sixteen percentage points and beating every baseline: logistic regression (0.5147), LightGBM (0.5286), and the imbalance rule (0.4970). At $h = 50$ and $h = 200$ it is no longer ahead: its 0.5312 and 0.5128 sit within about two thousandths of the strongest baselines (the imbalance rule's 0.5321 at $h = 50$, logistic regression's 0.5165 at $h = 200$), a statistical tie. So the summary is *best at the shortest horizon, competitive at longer ones*: the deep model adds value where the sequence structure is richest. The edge also holds month to month: `tcn_small` beats the per-month majority baseline in all three held-out months at every horizon (Figure 7; the per-month numbers at h=50 are tabulated in Appendix C, Table 20).

Balanced accuracy. Raw accuracy flatters the long horizons, where almost every event is up or down and a two-class guess scores near 0.5. Balanced accuracy, the mean of per-class recalls, tells the sharper story: `tcn_small` leads at $h = 10$ (0.486 versus 0.333 for the always-one-class `no_change`, and 0.479 for logistic regression), but balanced accuracy falls towards 0.34–0.37 at $h = 50$ and $h = 200$ for *all* models because the vanishing neutral class is almost never recalled. In other words, the genuine multi-class skill lives at the short horizon; the longer-horizon accuracy is a near-binary up/down problem.

The multi-task models. The execution-aware variants are weaker direction classifiers: `tcn_exec_base` and `tcn_exec_ret0` sit near 0.41 at $h = 10$, well below the baselines, because the shared trunk is optimised for return, quantile, markout and adverse heads simultaneously and the direction head is diluted. Adding the latent state-space context (`tcn_exec_ret0_ssm`) lifts $h = 10$ accuracy to 0.5071, nearly ten points, turning the multi-task model from clearly-worse than baseline into baseline-competitive. That gain is isolated and quantified in Section 10.2; it is the single most effective architectural change in the study.



(a) Out-of-sample R^2 by month

(b) Rank-IC by month

Figure 8: Out-of-sample R^2 and rank-IC by held-out month for the two return-emitting models.

Q1–Q2. Short-horizon directional signal exists and a compact causal TCN captures it: `tcn_small` is the best model at h=10 (0.5333) and ties the strongest baselines at longer horizons, and, the result the walk-forward was built to test, its edge holds in *all three* held-out months rather than a single one. Bigger is not the lever: the larger multi-task network needs the latent-state context to reach baseline-competitive direction accuracy.

8.2 Return signal

Point-return prediction is far harder than direction, and the results separate the two return-emitting models cleanly (Table 7). Ridge regression has a *genuine but tiny* short-horizon edge: a positive out-of-sample R^2 of 0.049 at $h = 10$ (26) and a rank information coefficient (25) of 0.31, both decaying monotonically to essentially zero by $h = 200$. The multi-task network's neural return head, by contrast, is unstable: its R^2_{oos} is strongly negative (roughly -31 at $h = 10$), meaning the point forecast is far *worse* than simply predicting the training-mean return, and its rank-IC is indistinguishable from zero.

Table 7: Out-of-sample R^2 and rank-IC by horizon for the two return-emitting models (mean over test folds).

	R^2_{oos}			rank-IC		
Model	h=10	h=50	h=200	h=10	h=50	h=200
`ridge_regression`	0.049	0.007	−0.004	0.308	0.111	0.034
`tcn_exec_base`	−30.8	−11.4	−2.9	0.000	0.009	−0.013

The practical reading is twofold. First, what point-return signal exists is small and concentrated at the shortest horizon, consistent with the sub-basis-point spreads of Section 4. Second, the multi-task return head is a liability: noisy enough to blow up R^2_{oos} and, as Section 9 shows, to drive catastrophic over-trading in the taker backtest. This is exactly why its loss weight was progressively reduced and finally removed (the `ret0` variant of Section 6.5), a decision the ablations of Section 10 justify. Ridge regression remains the only model with usable point-return skill.

8.3 Distributional calibration

The reason to carry quantile heads at all is to produce *calibrated* uncertainty that a quoting policy can act on. Table 8 reports the empirical coverage of the nominal-90% central interval (13) for the three multi-task variants, and Figure 9 breaks it out by month. Coverage lies in 0.84–0.95 across variants and horizons, close to the nominal 0.90 and stable across the three held-out months.

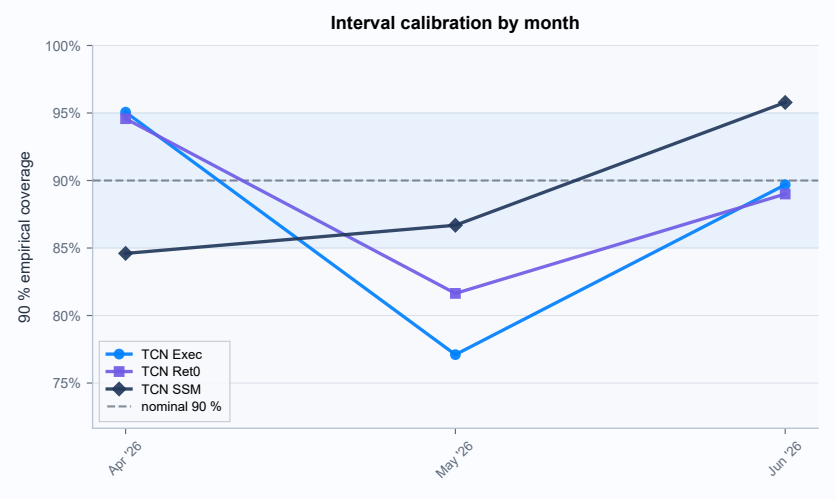


Figure 9: 90% interval coverage by held-out month, against the nominal 0.90 reference.

Table 8: Empirical 90% interval coverage by variant and horizon (mean over test folds).

Variant	h=10	h=50	h=200
`tcn_exec_base`	0.854	0.869	0.896
`tcn_exec_ret0`	0.954	0.842	0.857
`tcn_exec_ret0_ssm`	0.922	0.857	0.892

The coverage figures sit close to nominal and stay stable month to month, in marked contrast with the calibration that collapsed under naive early stopping in development (Section 10.3). That this calibration is both good *and* robust is a direct product of the composite validation score and two-phase training of Section 7.5: selecting on direction accuracy alone, as in early development, restored checkpoints at which the quantile head was under-trained and coverage collapsed toward 0.5; the dedicated calibration pass fixed it without a fragile dependence on an exact stopping epoch. Coverage also holds across the volatility, spread, and liquidity regimes of Section 4.5; the per-regime breakdown is deferred to Appendix C.

8.4 Execution-head accuracy and summary

The markout and adverse-selection heads are evaluated in two complementary ways. Their *calibration* as forecasts is captured by the interval coverage above and by their contribution to the composite selection score (27); their *decision value*, the question that actually matters, is measured directly in Section 9, where the policies consume them to decide whether and where to quote. A standalone markout/adverse error table was not exported in the walk-forward artefacts and is deferred to Appendix C.

To summarise the predictive picture against the opening sub-questions: **Q1**, yes, short-horizon directional signal exists and is robust across months, though the effect size is modest (a few points over a strong baseline at $h = 10$); **Q2**, a compact TCN matches or slightly beats the linear and gradient-boosted baselines on direction, but only at the short horizon, and the larger multi-task network is competitive only once the latent-state context is added; **Q3**, multi-task learning buys *calibrated intervals*, not a better direction classifier. Whether any of this skill survives execution costs (**Q6**) is the subject of Section 9.

9 From Prediction to Execution

Section 8 found a small predictive edge. The question for Q6 is whether that edge can be turned into orders that make money. I test it in two execution settings: aggressive trading after a threshold signal (Section 9.1) and passive quoting in a replay simulator (Sections 9.2 to 9.5). Both use only the held-out months, and every policy parameter is selected on validation and then frozen. These tests are the main stress test for the predictive results.

9.1 Taker sanity backtest

The first test is the simplest one: can a point-return forecast pay for itself if traded *aggressively*? At each event a directional signal is converted to a marketable order (buy if $\hat{r}_{t,h} > \theta$, sell if $\hat{r}_{t,h} < -\theta$), executed one event later by crossing the spread and marked at the horizon. With a per-unit fee rate ϕ and latency ℓ, the net profit of a round trip is

$$\mathrm{PnL}_t^{\mathrm{buy}} = \left(m_{t+h} - p_{t+\ell}^{a,1}\right) - \phi p_{t+\ell}^{a,1}, \qquad \mathrm{PnL}_t^{\mathrm{sell}} = \left(p_{t+\ell}^{b,1} - m_{t+h}\right) - \phi p_{t+\ell}^{b,1}, \tag{28}$$

the first bracket being the gross mark-to-mid move and the second the fee paid to cross. The threshold θ is chosen to maximise validation net PnL, frozen, and applied to the test month; only the return-emitting models are eligible. Table 9 reports the result at $h = 50$ with a 5 bps taker fee and one event of latency.

Table 9: Taker backtest on the held-out months (h=50, 5 bps taker fee, one-event latency; threshold selected on validation).

Model	Trades	Gross PnL	Net PnL	Hit rate
`no_change`	0	0	0	—
`ridge_regression`	803	+1,970	−26,740	0.125
`tcn_exec_base`	17,097	−12,712	−642,809	0.068

The reading is unambiguous. Ridge regression, the one model with genuine point-return skill (Section 8.2), is gross-positive (+1,970) but pays it all back and more in fees, ending at −26,740 over 803 trades with a 12.5% hit rate. The multi-task return head is worse on every axis: its noisy forecast crosses the threshold constantly, generating 17,097 trades that are gross-negative even before fees and net −642,809. This is the sanity check doing its job, and it directly motivates the return-head removal of Section 6.5.

Taker verdict. Short-horizon mid-price direction, even when predicted slightly better than chance, is *not* a taker-tradeable edge at these costs: no model clears a 5 bps fee, and the only model with positive gross PnL still loses decisively net of the spread it must cross.

9.2 The passive market-making simulator

The more demanding, and more appropriate, test is whether the forecasts improve *passive liquidity provision*, where one earns the spread instead of paying it. The simulator replays each held-out day from flat inventory, letting a policy post quotes every `decision_interval` = 200 events and accounting for the consequences. Figure 10 sketches the loop.

Fills. A resting quote either trades or expires over its horizon. The baseline fill rule is conservative: a bid posted at price p^q fills if the mid trades down to it,

$$\text{fill if } \min_{u\in(t,t+h]} m_u \le p^q \quad (\text{ask: } \max_{u\in(t,t+h]} m_u \ge p^q), \tag{29}$$

at the first such event, else it expires. The reported results use a stricter *queue-aware partial-fill* refinement of this rule, which adds a displayed-depth queue ahead of the quote and allows partial fills; it is defined and ablated in Section 10.4.

Accounting. A filled bid increases inventory and pays cash, a filled ask does the reverse, and wealth is marked to the mid:

$$\text{bid: } x \mathrel{+}= q,\ C \mathrel{-}= qp^{\mathrm{fill}} + \text{fee}; \quad \text{ask: } x \mathrel{-}= q,\ C \mathrel{+}= qp^{\mathrm{fill}} - \text{fee}; \qquad W_t = C_t + x_t m_t, \tag{30}$$

subject to a hard inventory limit $|x_t| \le x_{\max} = 1$ (a breaching quote is rejected and logged). The per-decision reward is (6) with weights $\lambda_{\mathrm{inv}} = 0.01$, $\lambda_{\mathrm{turn}} = 0.001$, $\lambda_{\mathrm{dd}} = 0$, $\lambda_{\mathrm{adv}} = 0.25$, and the *realised* adverse-selection cost of a fill is measured against the mid h events later,

$$A^{\mathrm{bid}} = \max\left(0,\, p^{\mathrm{fill,bid}} - m_{\tau+h}\right), \qquad A^{\mathrm{ask}} = \max\left(0,\, m_{\tau+h} - p^{\mathrm{fill,ask}}\right). \tag{31}$$

This is the quantity that punishes a liquidity provider for being run over, and, as the results show, it is decisive.

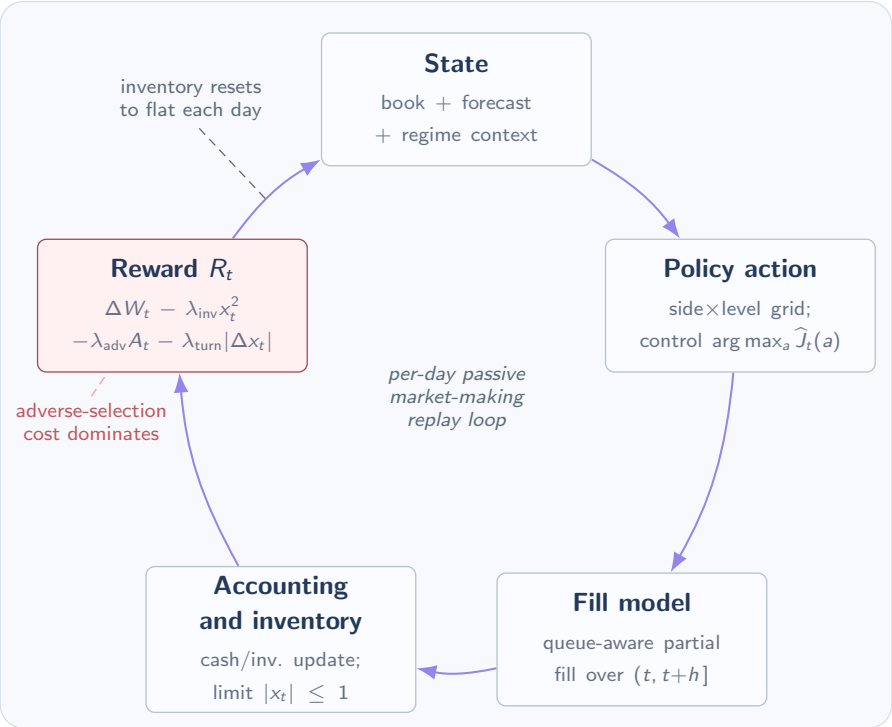


Figure 10: The market-making replay environment: state, quoting action, fills, accounting, and the per-decision reward.

9.3 Policies

Six policies are compared, spanning forecast-free baselines to the forecast-driven control optimiser (Table 10). Each policy's free parameters are grid-searched on validation total reward, frozen, and then evaluated on the test month; the test month is used once.[2]

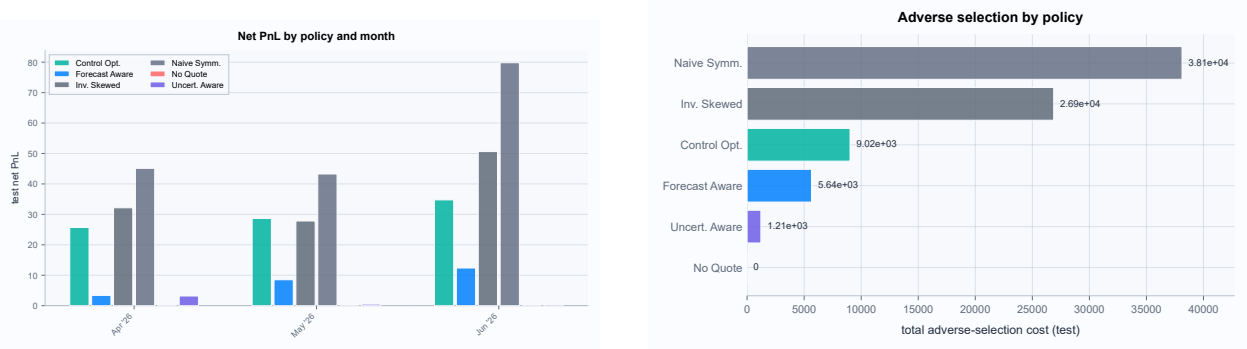
Table 10: Market-making policies from forecast-free baselines to the control optimiser; parameters selected on validation. `no_quote` never quotes and earns zero.

Policy	Quoting behaviour	Validation-selected grid
`no_quote`	never quotes (reference)	—
`naive_symmetric_mm`	always both sides	—
`inventory_skewed_mm`	both sides, skewed by inventory	soft limit $\rho \in \{0.25, 0.5, 0.75\}$
`forecast_aware_mm`	side(s) chosen by return sign	$\theta \in \{0, 10^{-5}, 5{\times}10^{-5}, 10^{-4}\}$
`uncertainty_aware_mm`	drops a side on wide interval / high adverse	$u_{\text{thresh}} \in \{0, 0.5, 1, 1.5, 2\}$
`control_quote_optimizer`	arg max over side×level grid	reward weights λ

9.4 Market-making results

Table 11 gives the policy comparison summed over the three held-out months, and Figure 11 shows net PnL and adverse-selection cost by policy. The pattern is stark and consistent: *every* quoting policy has negative total reward, and the ranking is governed almost entirely by adverse selection.

[2] A learned contextual-bandit policy was studied in the development campaign; it consistently overfit its validation selection (positive on validation, negative on test), and is discussed as a cautionary case in Section 10.



(a) Net PnL by policy & month

(b) Adverse-selection cost

Figure 11: Market-making diagnostics by policy: net PnL by month and adverse-selection cost.

Table 11: Market-making policy comparison summed over the three held-out months, driven by the `tcn_exec_base` forecasts.

Policy	Fills	Net PnL	Total reward
`no_quote`	0	0.00	**0.00**
`uncertainty_aware_mm`	164	8.15	−426.6
`forecast_aware_mm`	834	48.52	−8,284
`control_quote_optimizer`	1,404	178.21	−14,068
`inventory_skewed_mm`	4,184	221.23	−33,139
`naive_symmetric_mm`	6,324	336.40	−36,633

The forecast-free baselines quote indiscriminately: `naive_symmetric` fills 6,324 times and earns the most raw net PnL (+336), but precisely because they provide liquidity to everyone, they accumulate enormous adverse-selection cost (76,233), ending at −36,633. The forecast-using policies all learn the same lesson, which is to quote *less*. `uncertainty_aware` fills only 164 times, bleeds the least adverse cost, and posts the best (least negative) quoting reward of −427. In other words, the most useful thing the forecast tells a market maker here is mostly *not to provide liquidity*, a striking illustration of what we call the adverse-selection wall: the realised cost of being filled scales with volume and, at these horizons and spreads, always exceeds the spread captured.

9.5 The control optimiser and the anti-triviality check

The most sophisticated policy is a transparent, grid-based control optimiser (no reinforcement learning). At each decision it scores a nine-action grid of quote sides and book levels and picks the maximiser of an expected risk-adjusted reward, after rejecting any action that could breach the inventory limit under worst-case fills:

$$\widehat{J}_t(a) = \sum_{(s,\ell)\in S(a)} \hat{p}^{\,\text{fill}}\, q\left(\widehat{M}^{s,\ell} - \lambda_{\text{adv}}\widehat{A}^{s,\ell} - \text{fee}\right) - \lambda_{\text{inv}}\widehat{x}_{t+1}^2 - \lambda_{\text{turn}}\widehat{T}_t - \lambda_{\text{unc}}\widehat{U}_t - \lambda_{\text{act}}\mathbf{1}\{a \neq \varnothing\}, \qquad (32)$$

where $\hat{p}^{\,\text{fill}}$ is a per-fold fill-probability model, $\widehat{M}$ and $\widehat{A}$ are the level-adjusted markout and adverse-selection forecasts from the multi-task heads, and $\widehat{U}$ is the uncertainty score (21). It is given every advantage the model can supply.

To judge it fairly we add the `no_quote` policy (which never quotes and so earns exactly zero) as an anti-triviality reference, and declare the optimiser a successful market maker only if it beats `no_quote` *and* `naive_symmetric` in a majority of test months while clearing a minimum per-month fill-activity floor. Table 12 is the scorecard.

Table 12: Anti-triviality scorecard for the control optimiser over the three held-out months.

Criterion	Result
Beats `naive_symmetric_mm` (majority of months)	yes (3/3)
Clears the per-month fill-activity floor	yes (3/3)
Beats `no_quote` (majority of months)	**no** (0/3)
Classification	*risk-avoidance policy*

The control optimiser quotes actively (1,404 fills) and turns a positive raw net PnL (+178), and it beats the naive maker in all three months: selectivity clearly helps. But its realised adverse cost (18,040) still dominates, leaving a total reward of −14,068, and it fails to beat `no_quote` in a single month. The queue-aware fill model, the fill-probability model, and the control machinery, all built to give the forecasts their best possible chance, ran cleanly on real data but did not manufacture decision value that the simpler simulator was hiding.

Q6 — decision value. Across the three held-out months, the forecasts do not support a profitable execution policy under the stated assumptions. The taker strategy loses after a 5 bps fee, and no passive-quoting policy — including a control optimiser handed calibrated markout, adverse, fill-probability, and uncertainty signals — beats `no_quote` in any month. The cause is adverse selection: realised fill cost scales with quoting volume and exceeds the spread captured at these horizons and costs.

10 Ablations and Development

Sections 8 and 9 reported *what* the models do; this section isolates *which design choices mattered*. Each execution-facing addition is a module that can be toggled on its own, so its effect can be measured in isolation. Each has a fixed retain/drop rule, and I apply that rule even when the qualitative story is less neat. In brief: the latent state-space context is **retained** (it improves a paired no-SSM run); the zero-return-head change is tagged *drop* by the automated coverage rule, though the fuller evidence below favours removing the head; the control quote optimiser is classified *risk-avoidance only* (it never beats not quoting); and the queue-aware partial-fill model is retained as a diagnostic. The subsections expand each, and Section 10.3 records how the final design was reached.

10.1 Return-head ablation

Does the noisy point-return target harm the shared representation (Q4)? Comparing `tcn_exec_ret0` (return head removed) against `tcn_exec_base` (return weight 0.10) on the held-out months (Table 13), removing the return head *helps and stabilises* direction: `ret0` is at least as accurate as `base` at $h = 50$ (0.4969 vs 0.4904) and $h = 200$ (0.5125 vs 0.4979), and it beats the per-month majority baseline in all three months versus `base`'s two. It also removes the over-trading pathology entirely: with no point-return output there is simply nothing for the taker rule to act on, so the catastrophic −642,809 of Section 9.1 cannot occur.

Table 13: Multi-task variants: direction accuracy by horizon, months beating the per-month baseline at h=50, and mean 90% interval coverage.

Variant	h=10	h=50	h=200	Months beat (h50)	Coverage (mean)
`tcn_exec_base`	0.4077	0.4904	0.4979	2/3	0.873
`tcn_exec_ret0`	0.4101	0.4969	0.5125	3/3	0.884
`tcn_exec_ret0_ssm`	**0.5071**	**0.5192**	0.5023	3/3	0.890

The deterministic scorecard nonetheless tags `ret0` "drop". This is not a contradiction but a matter of scope: the automated rule compares only interval *coverage* against `base`, and on that single axis `ret0` is not clearly better. The fuller reading, which the report adopts, is that zeroing the return head modestly improves direction, is neutral-to-slightly-better on calibration, and eliminates over-trading, so removing it is the right call. This vindicates the progressive reduction of the return weight ($1.0 \to 0.25 \to 0.10 \to 0$) traced in Section 10.3.

10.2 Latent state-space context

The single most effective change in the whole study is the latent state-space context of Section 6.6. Because the SSM features are appended *per variant*, `tcn_exec_ret0_ssm` and `tcn_exec_ret0` are the same network trained on otherwise identical data, so their difference isolates the context's contribution. It is large at the short horizon: $h = 10$ accuracy rises from 0.4101 to 0.5071, nearly ten percentage points, with a further +2.2 points at $h = 50$ (Table 13), turning a clearly-below-baseline multi-task model into a baseline-competitive one. The deterministic rule retains it.

This result also carries a methodological lesson. On the earliest fold's validation month the SSM variant looked *worst*, and a single-fold study would have discarded it; only across the three held-out test months did the real, positive effect emerge. It is precisely the kind of finding the walk-forward protocol (Section 7) was built to surface and a single-split evaluation would have hidden.

10.3 Development: regularisation, early stopping, and the calibration arc

The networks reached their final form over four development iterations on the fixed split, a textbook sequence of over-/under-fitting diagnosis. The first iteration under-trained (three epochs, no regularisation); the second trained to full length but, with no dropout or weight decay, the larger kernel-7 / five-layer `tcn_small` *overfit* and regressed below its under-trained self, while the multi-task return head diverged. The third iteration was a redesign (a smaller `tcn_small` with kernel 5 and four layers, dropout and weight decay, a larger multi-task trunk, the return weight cut from 1.0 to 0.25, and early stopping) which made `tcn_small` the best direction model but, by stopping on direction accuracy alone, restored a checkpoint where the quantile head was under-trained and coverage collapsed. The fourth iteration loosened early stopping and cut the return weight further to 0.10, recovering calibration. The redesign was informed by a grid over depth ($\{3, 5\}$ layers), kernel size ($\{3, 5, 7\}$), and channel width ($\{16, 32, 64\}$), whose clearest signal was that *narrow* models generalised best: the 16-channel configurations topped the validation ranking and the 64-channel ones sat at the bottom, which is why the deployed `tcn_small` uses only 16 channels. The per-iteration hyperparameters and the interval-coverage arc through development (well-behaved in the first two iterations, a collapse toward 0.5 in the third, a partial recovery in the fourth) are tabulated in Appendix D. The walk-forward evaluation removed the fragility entirely by replacing the hand-tuned minimum-epoch fix with the principled composite validation score and two-phase calibration pass of Section 7.5, yielding the stable 0.84–0.95 coverage of Section 8.3.

10.4 Queue-aware partial-fill model

The final addition tests whether the "no decision value" verdict is an artefact of a crude fill model. Top-5 snapshots carry no true queue position, but a stricter conservative proxy can be built from displayed depth. A resting quote at price p^q starts with a queue ahead of it, $Q_t^{\text{ahead}} = \rho_q Q_t(p^q)$ with $\rho_q \in \{0.25, 0.5, 1.0\}$, and over its horizon accumulates effective depletion; it begins to fill only once that depletion exceeds the queue ahead:

$$D_{t,u}(p^q) = \sum_{v=t+1}^{u} \kappa \max\big\{0,\, Q_{v-1}(p^q) - Q_v(p^q)\big\}, \qquad q_u^{\text{fill}} = \min\big(q^o,\, \max\{0,\, D_{t,u}(p^q) - Q_t^{\text{ahead}}\}\big), \tag{33}$$

where κ is the fraction of displayed depletion treated as fill-relevant; a full fill is assumed only if the opposite best trades through p^q. The fraction κ and the queue position are selected on validation, with one fill model per fold for all policies. The model behaves plausibly on real data: partial fills occur, average fill fractions are high but below one, and fill rates vary sensibly with how selectively each policy quotes (the per-policy fill diagnostics are in Appendix C, Table 21).

The fill model produces non-pathological fills, so it is retained as a diagnostic. It did not change the verdict. The queue-aware fill model, the fill-probability model, and the control optimiser of Section 9.5 were each added to give the forecasts a better chance, and the "no decision value" conclusion held under all of them. This matters for how the result should be read: it is not that the simulator was too crude to find an edge. The later simulator is more favourable to the forecasts than the conservative single-touch rule, and the edge still does not appear.

11 Engineering, Reproducibility, and Testing

Because the main empirical result is negative, reproducibility matters: the conclusion is only as good as the guarantee that a pipeline bug did not manufacture or mask an edge. This section records the implementation choices that make the study run on a single workstation and reproduce exactly, and the tests used to check the leakage invariants.

11.1 Performance and scale

Seven optimisations (Table 14) make a multi-hour, 1.6M-row walk-forward tractable on commodity hardware. The two with the largest leverage are the vectorised snapshot reconstruction (which turned a 1272 s per-day Python loop into a 4 s NumPy reshape, verified column-for-column identical to the reference engine) and selective retraining with prediction reuse, which lets a campaign retrain only the models that changed and copy the rest's frozen prediction files, the single biggest wall-time saving across the development iterations.

Table 14: Performance and scale optimisations and their effect, each local to one pipeline stage.

Optimisation	**Effect**
Vectorised snapshot reconstruction	1272 s $\rightarrow$ 4 s per day ($\sim$300$\times$), validated identical
Event-time striding, once at the book stage	one shared sampled stream; $\sim$10$\times$ smaller everything downstream
Vectorised sequence-window construction	$O(1)$ prefix-sum window admission (Section 5.4)
Periodic market-making decisions (200 events)	full-day replay made tractable
Column-pruned market-making frame	simulator loads only the $\sim$18 columns it needs
Per-stage / per-epoch progress reporting	visibility and timing for long runs
Selective retraining + prediction reuse	retrain only changed models; largest wall-time saving

11.2 Reproducibility and run artefacts

Every run is written fully self-contained under its own run identifier, so independent runs coexist on disk and are compared directly without re-execution:

```
artefacts/runs/{run_id}/
  resolved_config.yaml                 # the frozen, fully-resolved configuration
  transforms/{label_thresholds,regime_thresholds}.yaml
  metrics/monthly_*.parquet            # predictive / distributional / robustness
  folds/{fold}/
    dataset_metadata.yaml              # the fold's split definition
    transforms/scaler.pkl              # scaler fit on this fold's train rows
    models/{model}/{model.bin, train_ckpt.pt}
    predictions/{model}.parquet        # the long prediction contract (Section 6.3)
    logs/{model}.jsonl                 # one structured line per epoch
    market_making/{orders,fills,inventory,policy_metrics}.parquet
```

The frozen `resolved_config.yaml` records every resolved parameter, so a run is fully described by its configuration plus the immutable, checksummed raw inputs (Section 5.1). Because predictions are written in the single self-describing long schema of Section 5.3, one run's frozen predictions can be re-evaluated or re-backtested by another without retraining: the mechanism behind selective retraining above.

11.3 Crash-resumability

A full walk-forward with the multi-task variants is a multi-hour job, so the driver is crash-resumable at three granularities. At the *model* level, a model whose prediction file already exists is skipped and loaded, and finished folds are skipped wholesale. At the *epoch* level, the multi-task model writes an atomic per-epoch checkpoint (network state, phase index, epoch-in-phase, best score and state, no-improve counter, and training log), so on resume it restores and continues from the next epoch, and a crash costs at most one epoch. At the *dataset* level, an existing fold's split definition, scaler, and latent-state context are reused rather than rebuilt. Driver flags (`--folds`, `--variants`, `--phase-a-epochs` / `--phase-b-epochs`, `--mm-model`) let a long run be fitted to a compute budget without code changes.

11.4 Testing discipline

The suite has **267 tests**, organised one file per stage, and targets the invariants the results depend on (Table 15). The leakage rules of Section 5.4 are not merely intended but *tested*: that no sequence window or label crosses a monthly day, that every fitted transform sees only training rows, that the multi-task save/load reproduces predictions and its quantiles stay monotone, and that policy selection touches only the validation split.

Table 15: What the 267-test suite guarantees by component; leakage-prevention invariants are explicitly asserted.

Component	What is asserted
Monthly splits	reject non-BTCUSDT / `top_k` $\neq 5$ / non-first-of-month; no window or label crosses a day
Regime & label thresholds	fitted on training rows only
Label mathematics	quantile / markout / adverse formulas hand-checked
Multi-task model	save/load reproduces predictions; quantiles monotone; adverse ≥ 0; loss masks unavailable labels
Return-head ablation	stop-gradient leaves the encoder's gradients unchanged
Composite selection score	computed on validation rows only
Market-making	conservative & queue-aware fills; inventory-limit rejection; validation-only policy selection
Checkpoint resume	an epoch checkpoint restores and continues training faithfully

12 Discussion, Limitations and Conclusion

This section maps the results back onto the six sub-questions of Section 1.2. Table 16 states each answer with its evidence and its implication; the subsections that follow expand the rows that carry the argument.

Table 16: Summary of the six research sub-questions with evidence, verdict, and implication.

Sub-question	Evidence	Verdict	Implication
Q1 Directional signal beyond baselines?	§8.1	yes (modest)	Short-horizon signal is real and robust, but only a few points.
Q2 Deep models beat linear / GBM?	§8.1	partial	Best at h=10 only; a statistical tie at longer horizons.
Q3 Multi-task helps the execution heads?	§8.3	yes / no	Buys calibrated intervals, not a better direction classifier.
Q4 Point-return head: help or harm?	§8.2, §10.1	harms	Noisy; wrecks R^2_{oos} and drives over-trading; remove it.
Q5 Latent state-space context helps?	§10.2	yes	+9.7pp at h=10; the most effective single change.
Q6 Signal survives execution?	§9	no	No taker edge; no quoting policy beats not quoting.

12.1 Predictive skill exists, narrowly

There *is* short-horizon directional structure in the top-5 book, and a compact causal TCN captures it: `tcn_small` is the best model at $h = 10$ and clears the per-month majority baseline in all three held-out months (Q1). Two qualifiers bound this. The edge over a *strong* baseline is small, a couple of percentage points at the short horizon, and it narrows to a statistical tie with logistic regression and the imbalance rule at $h = 50$ and $h = 200$ (Q2). And the lever that closed the multi-task model's gap was not capacity but *context*: doubling the channels did nothing for direction, whereas the four-state latent filter added nearly ten points at $h = 10$ (Q5). The takeaway for a modelling programme is that, on data this weakly predictable, architectural *interface* and *conditioning* buy more than raw size.

12.2 The return head was the right thing to remove

The point-return target was a persistent liability (Q4). Its loss weight fell across development from 1.0 to 0.25 to 0.10 and finally to zero, and every step was warranted: the neural return head is so noisy that its out-of-sample R^2 is large and negative (worse than predicting the training mean), its rank-IC is indistinguishable from zero, and, most damaging, it drives the catastrophic over-trading of the taker backtest (Section 9.1). Removing it improved direction and eliminated that pathology (Section 10.1). Point-return prediction is not worthless in principle: ridge regression retains a small, genuine short-horizon edge ($R^2_{\text{oos}} = 0.05$, rank-IC 0.31 at $h = 10$); but a single shared trunk forced to also emit a calibrated direction and interval is the wrong place to chase it.

12.3 Calibration is trainable, but was fragile

The quantile heads answer Q3 in two parts: multi-task learning does *not* make a better direction classifier, but it does produce well-calibrated uncertainty: 0.84–0.95 coverage against a nominal 0.90, stable across months. The important lesson is how nearly that was lost. Selecting checkpoints on direction accuracy alone restored a model whose slow

quantile head was under-trained, collapsing coverage toward 0.5 (Section 10.3); the fix was not more data but a better *objective* for selection, the composite validation score and the dedicated two-phase calibration pass (Section 7.5), which made good calibration the default rather than a fortunate by-product of an exact stopping epoch.

12.4 Where the signal fails in execution

The main result of the report is on this gap (Q6). A classifier can improve direction accuracy without producing orders whose expected markout exceeds fees and adverse selection, and that is what happens here. Table 17 tracks the same forecast through the stages of Figure 1: it improves held-out accuracy and the intervals are calibrated, but once it must be traded, no taker rule clears a 5 bps fee and no passive-quoting policy beats `no_quote`. This holds even for the control optimiser, which is handed calibrated markout, adverse, fill-probability, and uncertainty signals.

The mechanism is adverse selection. The realised cost of being filled scales with quoting volume and, at these horizons and sub-basis-point spreads, exceeds the spread captured. I do not read this as an artefact of an overly crude simulator. The queue-aware fill model, the fill-probability model, and the control optimiser are each more favourable to the forecasts than the conservative single-touch rule, and the no-quote baseline still dominates. The conclusion is therefore about the signal and the costs at these horizons, not about the fill rule.

Table 17: The signal-decay ledger: the same forecast viewed at four stages of execution.

Stage	Best available evidence	Survives?
Raw predictive signal	`tcn_small` h=10 acc. 0.533 vs majority 0.374; 3/3 months	yes
Calibrated uncertainty	90% interval coverage 0.84–0.95	yes
Taker edge (5 bps)	best net PnL $-26{,}740$ (ridge)	**no**
Passive-quoting edge	best quoting reward -427; `no_quote` $= 0$	**no**

12.5 Methodological takeaways

Beyond the specific verdict, the study reinforces four working principles. First, *separate predictive skill from decision value* and report them with different instruments; conflating them is how a sub-percentage-point classification edge gets mistaken for an alpha. Second, *strong baselines are not optional*: a deep model that beats a coin flip but not logistic regression has demonstrated nothing, and only the imbalance rule and the linear models made the TCN's small win legible. Third, *walk-forward beats a single split*: the latent-state context looked worst on one validation month and best across three test months, so a single-split study would have discarded the best idea in the report. Fourth, a negative result can stand on its own when the pipeline supports it. The leakage invariants of Section 5 are tested (Section 11), and the execution test was made more favourable to the forecasts, not less; so "no robust decision value" is a statement about this problem at these costs, not a symptom of a broken evaluation.

12.6 Limitations

The study is a research platform, not a trading system, and several scope choices bound its conclusions. The evaluation uses twelve first-of-month days with three held-out test months: stronger than a single split (it is what let the latent-state result of Section 10.2 be distinguished from a one-month fluke), but still a handful of regime snapshots, not continuous history, so the cross-month standard deviations are indicative rather than tight, and only three of the five available folds were run. The study also operates on a thinned, event-strided stream, so a horizon of 50 events is roughly 500 raw updates and the results describe the strided stream, not every micro-update. Fills remain approximate: top-5 snapshots carry no true queue position, and although the queue-aware partial-fill model of Section 10.4 adds queue-ahead, depletion, and partial fills, it is still a proxy, with the taker backtest only a single-touch sanity check; modelling true queue priority would need an order-level depth-diff feed. The decision modules are small (a four-state linear-Gaussian filter and a grid-based action search, not a learned policy), and the control optimiser ends up as a *risk-avoidance* policy, though the adverse-selection cost of Section 9 suggests the ceiling is low at these horizons and costs anyway. Finally, the scope is a single symbol, a single venue, and top-5 depth: no cross-sectional or cross-venue generalisation is tested, and no live-trading profitability is claimed.

12.7 Future work

Each pipeline stage is replaceable without disturbing the rest, which is the point of the staged architecture of Section 5. Ordered by expected information per unit of effort, the natural next steps are: run all five folds and extend beyond twelve months to tighten the cross-month intervals; give the deployed `tcn_small` the latent-state context directly, since

the context is the one component that demonstrably helped and `tcn_small` is the best direction model; feed the filtered state into the *sequence* rather than only the context branch, or replace the linear filter with a richer one; run the detached and ridge-sidecar return-head variants at scale; add an order-level feed for true queue priority; and, mindful of the adverse-selection ceiling, try a larger but disciplined learned quoting policy with held-out selection.

12.8 Conclusion

This report asked whether compact sequence models on top-5 BTCUSDT limit order book snapshots can produce calibrated, execution-relevant forecasts that survive temporal distribution shift, and whether those forecasts improve passive market-making decisions under realistic costs. The answer separates into two parts. Short-horizon directional signal *does* exist and a compact causal TCN captures it: it is the best model at the shortest horizon and beats the per-month majority baseline in all three held-out months, while the multi-task network produces well-calibrated 90% intervals. The signal is small, and at longer horizons it is a statistical tie with strong linear baselines; the lever that helped the multi-task model was a latent state-space context, not extra capacity.

The second part is the gap between prediction and decision, and this is where the study is most decisive. No model clears a 5 bps taker fee, and no forecast-driven passive-quoting policy — including a control optimiser given calibrated markout, adverse-selection, fill-probability, and uncertainty signals — beats simply not quoting, because realised adverse selection exceeds spread capture at these horizons and costs. The execution test was made more favourable to the forecasts over development, not less, and the leakage invariants are tested, so the result is narrower than "the model fails": the model forecasts direction slightly better than the baselines, but that edge is not large enough to support the execution policies tested here.

The value of the study is that one pipeline shows both outcomes in the same framework: a small predictive edge in the classification task, and no robust gain once the forecasts are traded. Every artefact is kept so the next iteration can start from the same evidence.

Verdict. The report finds a small predictive edge, but not a tradable one under the stated assumptions. A compact TCN beats the linear baselines on mid-price direction across all three held-out months and the multi-task model is well calibrated; yet nothing clears 5 bps taker fees, and no forecast-driven passive-quoting policy beats `no_quote` in the held-out execution tests.

A Feature and Label Details

Table 18 is the full causal feature catalogue. Per-feature summary statistics for a representative subset appear in Table 1 (Section 4.3), and per-horizon label availability and class balance in Table 2 (Section 4.6).

B Module Architecture and Data Contracts

The four core data contracts are summarised in Table 4 (Section 5.3). The configuration loader validates a run once into a typed object and refuses any configuration violating the study's invariants:

```
data.symbols != ["BTCUSDT"]                              -> error
data.top_k   != 5                                        -> error
sampling.mode != "event_time"                            -> error
embargo_events < max(horizons_events)                    -> error
#dates < min_train + validation + test  (months)         -> error
market_making.enabled and not include_markout_targets    -> error
contextual_bandit_mm requested with no validation fold   -> error
a monthly date that is not the first of its month        -> error
output paths escaping the project root                   -> error
event_stride < 1                                         -> error
```

The canonical walk-forward configuration, in abbreviated form:

```
data:    venue=binance  symbols=[BTCUSDT]  top_k=5
         monthly_snapshot.dates = 2025-07-01 ... 2026-06-01  (12)
sampling: mode=event_time  horizons_events=[10,50,200]
          feature_lookbacks_events=[10,50,200]  event_stride=10
splits:   mode=expanding_monthly_snapshot  6 train / 1 val / 1 test
          step_months=1  embargo_events=200
labels:   direction_threshold_alpha=0.5  quantiles=[0.05,0.50,0.95]
          include_markout_targets=true  include_adverse_selection_targets=true
datasets: sequence_length=100  exclude_crossed=true
market_making: fill_model=queue_aware_partial  decision_interval=200
          lambda_inv=0.01  lambda_turn=0.001  lambda_adv=0.25
backtest: horizon=50  fee_bps_taker=5.0  latency_events=1
```

C Full Results

This appendix collects the per-fold and per-month tables kept out of the main body. The headline accuracy and return tables are in Section 8 (Tables 6, 7, and 8); per-class confusion matrices for the walk-forward run remain to be regenerated and are omitted here rather than shown from a superseded run. Full per-fold metric tables are written to each run's `metrics/` directory (Section 11.2).

Tables 19–20 give evaluation row counts and per-month accuracy. Tables 21 and 22 report fill diagnostics and regime-conditional interval coverage; calibration holds across regimes but degrades in the highest-volatility bucket, as expected.

D Hyperparameters and the Development Campaign

Table 23 gives the final (walk-forward) architecture and training settings for the two networks; the tabular baselines are specified in Table 5 (Section 6.1), and the composite selection score in (27).

For the `ret0` variant the return weight ω_r is set to zero; the latent-state variant additionally appends the four filtered states and their variances to the context branch (Section 6.6). Phase B freezes the encoder, pooling, context embedding, and fusion MLP, training only the calibration heads at a reduced learning rate with the direction weight set to zero.

The networks reached their final form over four development iterations on the fixed single-month split (Section 10.3). Tables 24 and 25 record the lever changed at each iteration and the interval-coverage collapse-and-recovery it produced; Tables 26, 27, and 28 give the per-iteration hyperparameters and the test accuracy each produced.

Table 18: Causal feature catalogue; all features use information at or before event t, computed per day.

Group	Feature(s)	Lookbacks (events)	Definition
Price / spread	m, s, s^{rel}, μ	—	(2), (7)
Imbalance	$I^{(1)}$, $I^{(K)}$	—	(8)
Order-flow imbalance	$\mathrm{OFI}_{t,L}$	$\{10, 50, 200\}$	(9)
Lagged returns	ret_L	$\{10, 50, 200\}$	(10)
Realised volatility	$\sigma_{t,W}$	$\{50, 200, 1000\}$	(10)
Raw levels	bid/ask price & size, levels 1–5	—	carried through
Regime descriptors	$\rho^{\mathrm{vol}}, \rho^{\mathrm{spr}}, \rho^{\mathrm{dep}}$ & buckets	1000	(11), (12)
Context	vol / spread / liquidity / time-of-day one-hot $\mathbf{c}_t$	—	Section 4.5

Table 19: Evaluation rows per split for the three folds (h=50, after embargo and label-availability exclusions).

Fold (test month)	Train rows (n months)	Validation rows	Test rows
`fold_2` (2026-04)	1,029,540 (8)	123,292	127,256
`fold_3` (2026-05)	1,152,832 (9)	127,256	121,727
`fold_4` (2026-06)	1,280,088 (10)	121,727	197,318

Table 20: Per-month test accuracy at h=50 for the strongest models.

Model	2026-04	2026-05	2026-06
`tcn_small`	0.5285	0.5179	0.5472
`logistic_regression`	0.5296	0.5140	0.5496
`imbalance_rule`	0.5300	0.5168	0.5494
`tcn_exec_ret0_ssm`	0.5145	0.5070	0.5361
`tcn_exec_base`	0.4979	0.4888	0.4845

Table 21: Queue-aware fill diagnostics on the held-out months; fill fractions and rates are per-month means.

Policy	Fills	Partial fills	Mean fill fraction	Bid / ask fill rate
`naive_symmetric_mm`	6,324	144	0.99	0.70 / 0.70
`forecast_aware_mm`	834	22	0.99	0.44 / 0.28
`control_quote_optimizer`	1,404	194	0.89	0.86 / 0.81

Table 22: 90% interval coverage of `tcn_exec_ret0_ssm` by market regime (h=50, mean over held-out months; nominal 0.90).

Regime dimension	Bucket	Coverage
Volatility	low	0.935
Volatility	medium	0.919
Volatility	high	0.800
Liquidity	thin	0.862
Liquidity	normal	0.852
Liquidity	deep	0.826
Spread	wide	0.857

Table 23: Final model and training hyperparameters used in the walk-forward evaluation.

Hyperparameter	`tcn_small`	`tcn_exec_multitask`
Channels	16	32
Kernel size	5	5
Layers (dilations)	4 $(1, 2, 4, 8)$	5 $(1, 2, 4, 8, 16)$
Dropout	0.10	0.10
Weight decay	5×10^{-4}	5×10^{-4}
Learning rate	5×10^{-4}	5×10^{-4}
Context / fusion hidden	—	32 / 64
Objective weights $(\omega_r, \omega_d, \omega_q, \omega_m, \omega_a)$	— (direction only)	$(0.10, 1.0, 1.0, 0.5, 0.25)$
Optimiser	AdamW	AdamW
Training schedule	early-stopped	two-phase (A ≤ 8, B 4 epochs)

Table 24: The four development iterations and what each established.

Iter.	**Principal change**	**Effect**
I	3 epochs, no early stopping, no regularisation	under-trained
II	full-length training, early stopping on	`tcn_small` overfits and regresses; return head diverges
III	smaller `tcn_small` (k5, 4 layers) + dropout + weight decay; return weight $1.0 \rightarrow 0.25$	`tcn_small` best direction; multi-task coverage collapses (stopped too early)
IV	looser early stopping; return weight $0.25 \rightarrow 0.10$	`tcn_small` gains hold; calibration recovers

Table 25: 90% interval coverage through development and the final walk-forward, for the multi-task model (mean over held-out months).

Stage	h=10	h=50	h=200
Iteration I	0.870	0.518	0.853
Iteration II	0.941	0.764	0.971
Iteration III	0.494	0.527	0.512
Iteration IV	0.693	0.910	0.763
Walk-forward (final)	0.854	0.869	0.896

Table 26: `tcn_small` hyperparameters across the four development iterations (channels fixed at 16).

Parameter	**I**	**II**	**III**	**IV**
Kernel size	7	7	5	5
Layers	5	5	4	4
Dropout	0	0	0.10	0.10
Weight decay	0	0	5×10^{-4}	5×10^{-4}
Learning rate	10^{-3}	10^{-3}	5×10^{-4}	5×10^{-4}
Max epochs	3	10	6	8
Early stopping	off	off	on	on (looser)

Table 27: `tcn_exec_multitask` hyperparameters across the four development iterations.

Parameter	**I**	**II**	**III**	**IV**
Channels	16	16	32	32
Kernel size	7	7	5	5
Dropout	0.05	0.05	0.10	0.10
Weight decay	10^{-4}	10^{-4}	5×10^{-4}	5×10^{-4}
Learning rate	10^{-3}	10^{-3}	3×10^{-4}	5×10^{-4}
Return weight ω_r	1.0	1.0	0.25	0.10
Early stopping	off	on	patience 3	patience 7, min 10

Table 28: Test direction accuracy through development (fixed single-month split).

Model	**Iter.**	h=10	h=50	h=200
`tcn_small`	I	0.505	0.538	0.497
`tcn_small`	II	0.479	0.510	0.509
`tcn_small`	III	0.523	0.545	0.512
`tcn_small`	IV	0.523	0.545	0.512
`tcn_exec_multitask`	I	0.366	0.486	0.483
`tcn_exec_multitask`	II	0.359	0.477	0.483
`tcn_exec_multitask`	III	0.368	0.484	0.490
`tcn_exec_multitask`	IV	0.364	0.484	0.478

References

Martin D. Gould, Mason A. Porter, Stacy Williams, Mark McDonald, Daniel J. Fenn, and Sam D. Howison. Limit order books. *Quantitative Finance*, 13(11):1709–1742, 2013.

Jean-Philippe Bouchaud, Julius Bonart, Jonathan Donier, and Martin Gould. *Trades, Quotes and Prices: Financial Markets Under the Microscope*. Cambridge University Press, 2018.

Lawrence R. Glosten and Paul R. Milgrom. Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1):71–100, 1985.

Rama Cont, Arseniy Kukanov, and Sasha Stoikov. The price impact of order book events. *Journal of Financial Econometrics*, 12(1):47–88, 2014.

Sasha Stoikov. The micro-price: a high-frequency estimator of future prices. *Quantitative Finance*, 18(12):1959–1966, 2018.

Zihao Zhang, Stefan Zohren, and Stephen Roberts. DeepLOB: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019.

Lorenzo Lucchese, Mikko S. Pakkanen, and Almut E. D. Veraart. The short-term predictability of returns in order book markets: A deep learning perspective. *International Journal of Forecasting*, 40(4):1587–1621, 2024.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling, 2018.

Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. WaveNet: A generative model for raw audio, 2016.

Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.

Marcos López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.

Dimitris N. Politis and Joseph P. Romano. The stationary bootstrap. *Journal of the American Statistical Association*, 89 (428):1303–1313, 1994.

Roger Koenker and Gilbert Bassett. Regression quantiles. *Econometrica*, 46(1):33–50, 1978.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.

Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82 (1):35–45, 1960.

Zoubin Ghahramani and Geoffrey E. Hinton. Parameter estimation for linear dynamical systems. Technical Report CRG-TR-96-2, University of Toronto, 1996.

Thomas Ho and Hans R. Stoll. Optimal dealer pricing under transactions and return uncertainty. *Journal of Financial Economics*, 9(1):47–73, 1981.

Marco Avellaneda and Sasha Stoikov. High-frequency trading in a limit order book. *Quantitative Finance*, 8(3): 217–224, 2008.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.